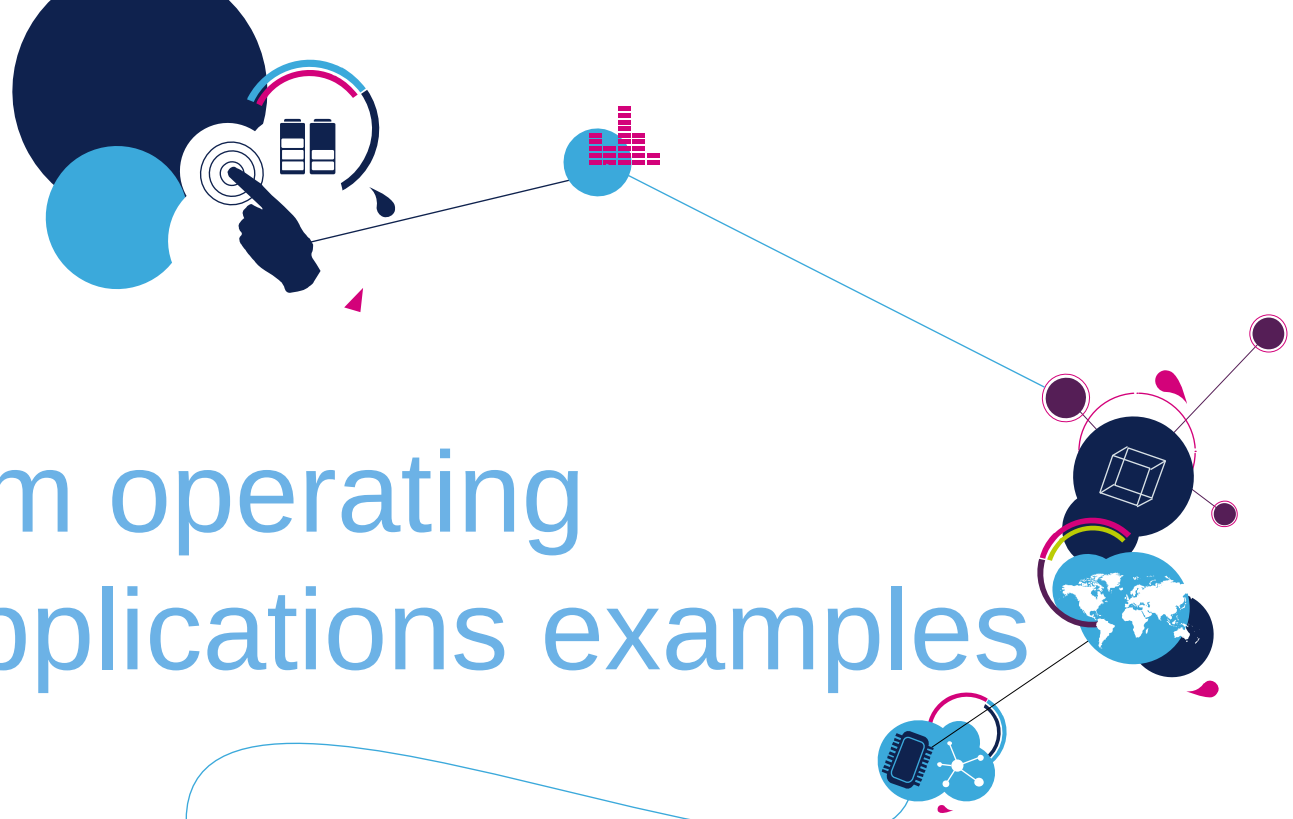
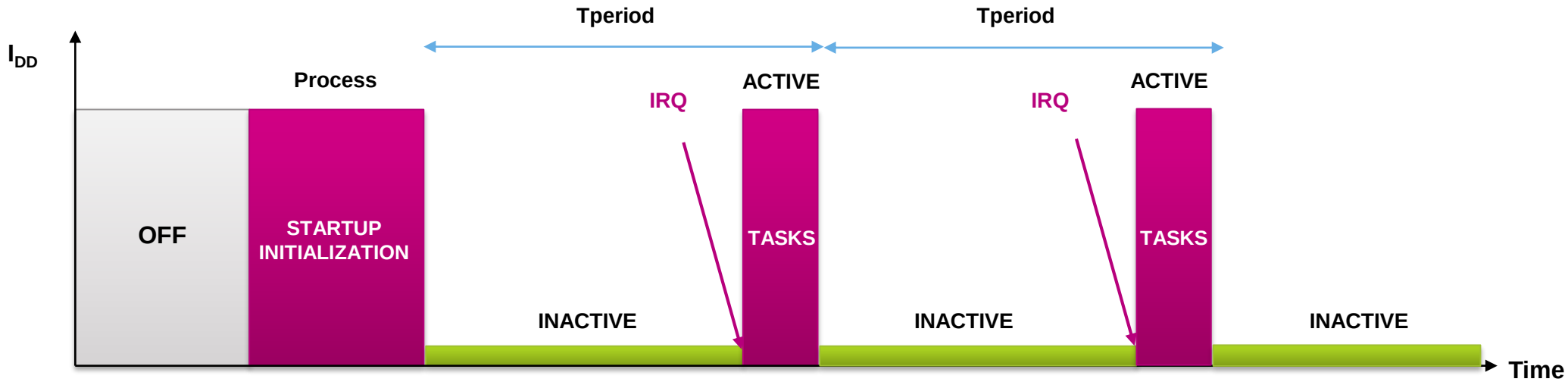


STM32L4 – System operating modes with real applications examples

life.augmented

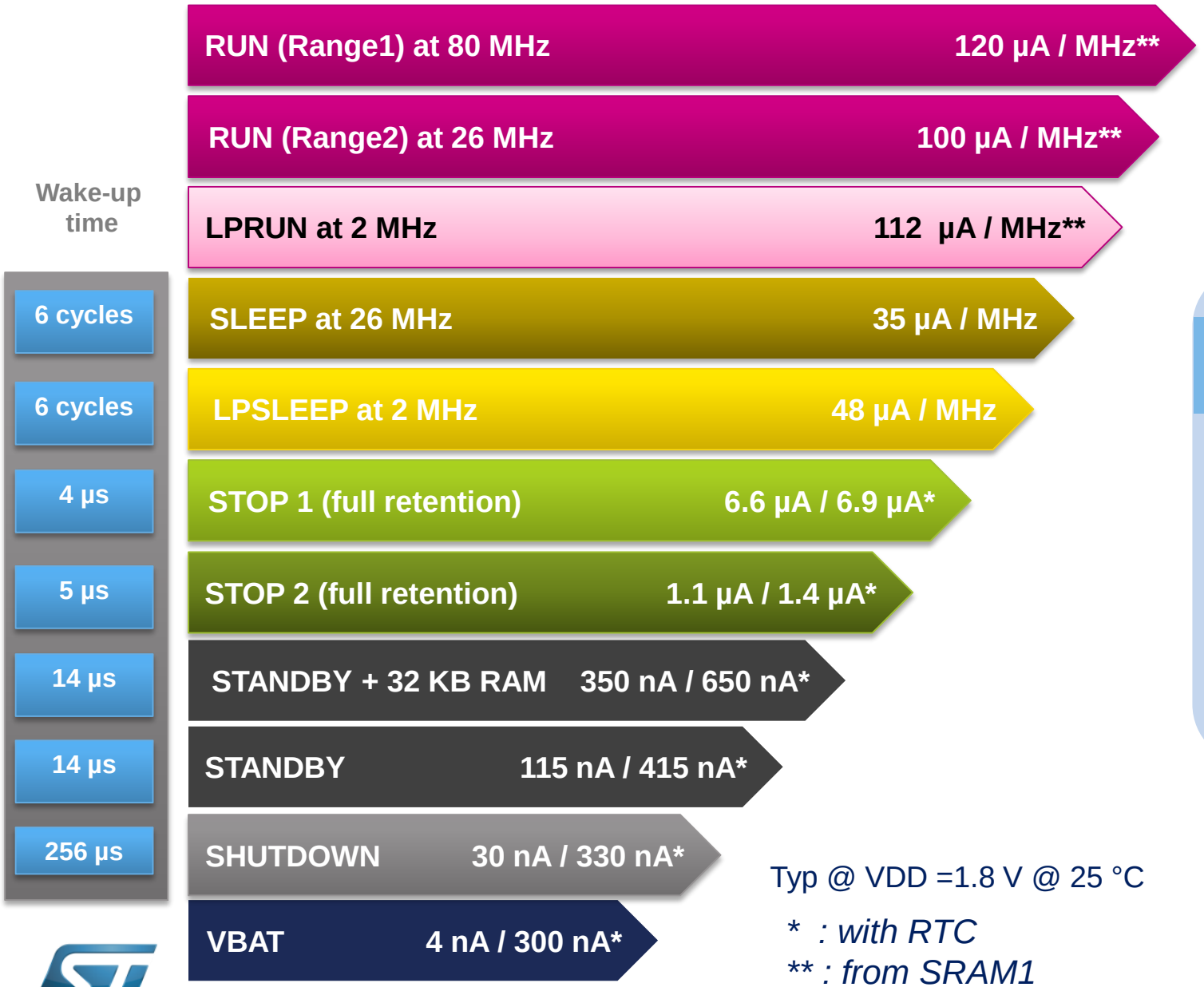
Typical application profile



- Application phases:
  - OFF – power is not applied to MCU
  - STARTUP INITIALIZATION – MCU performs configuration (peripherals, clocks, …)
  - Tperiod
    - INACTIVE – MCU is in low power mode to reduce power consumption
    - ACTIVE – MCU is in normal mode and performs tasks

life.augmented

Overview



Typ @ VDD =1.8 V @ 25 °C

** : with RTC*

*** : from SRAM1*

Application benefits

- High performance
  - ➔ CoreMark score = **273**
- Outstanding power efficiency
  - ➔ ULPBbench score = **187.7**

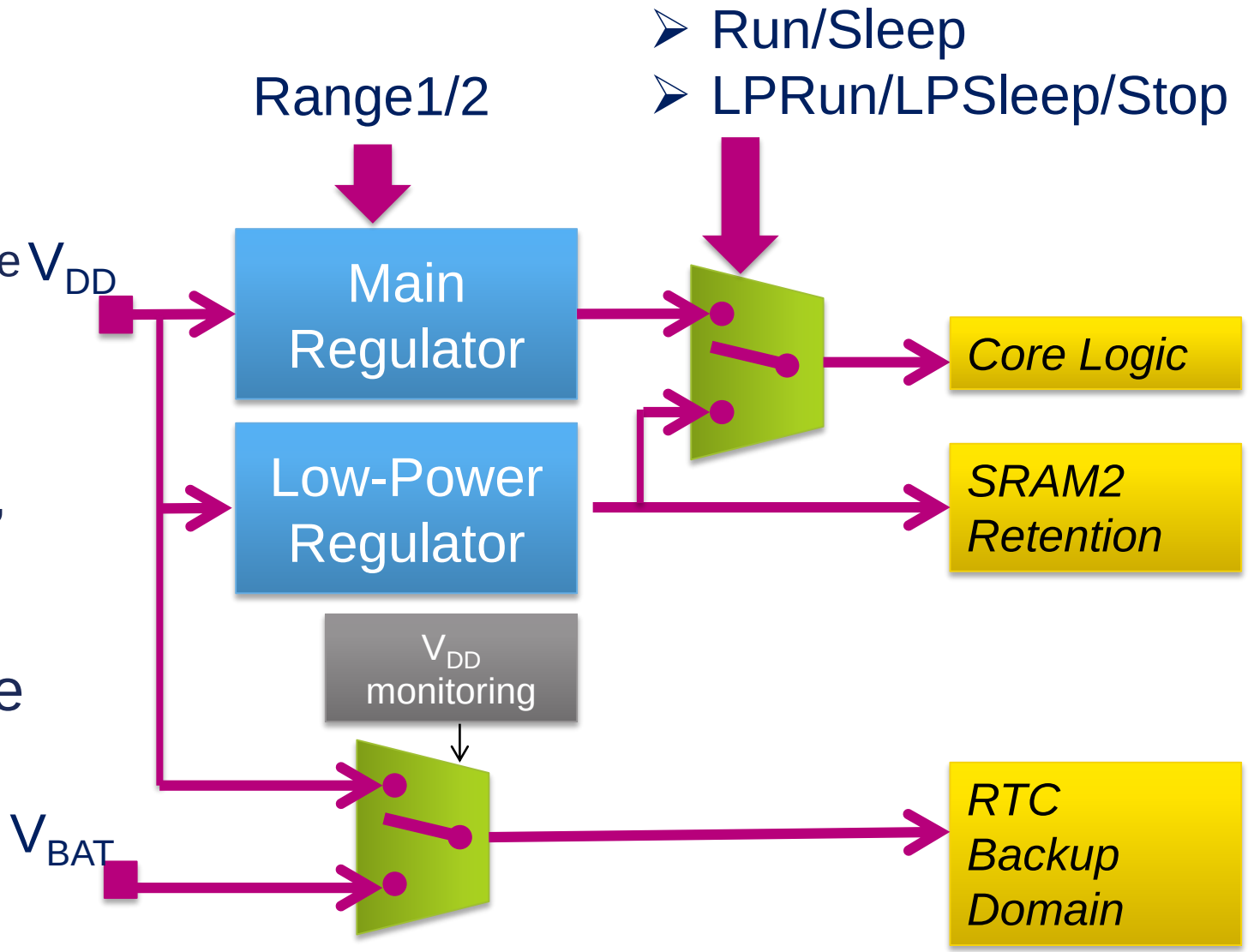
life.augmented

Key features

- Down to **100 µA/MHz** in Run mode with code execution from FLASH (RUN)
- Down to **350 nA** with 32 kB RAM retained (Standby)
- Down to **30 nA** with I/O wake-up (Shutdown)
- Wake-up from high number of peripherals (RTC in each mode)

Voltage regulators

- Two Voltage Regulators
  - The **Main Regulator** with two voltage ranges for Dynamic Voltage Scaling; used in Run and Sleep modes
  - The **Low-power Regulator** for Low-power run, Low-power sleep, Stop 1, and Stop 2 modes
- In Standby and Shutdown mode both regulators are off.



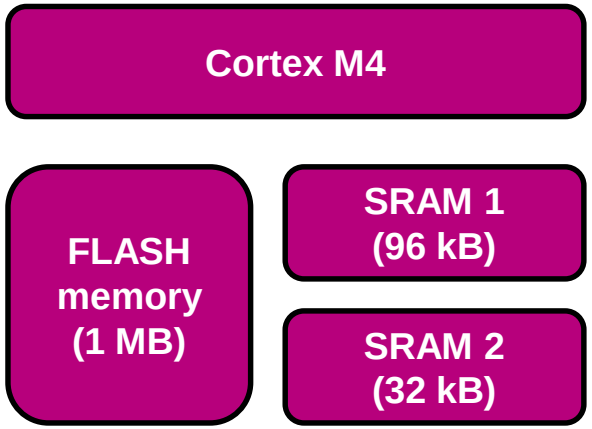
life.augmented

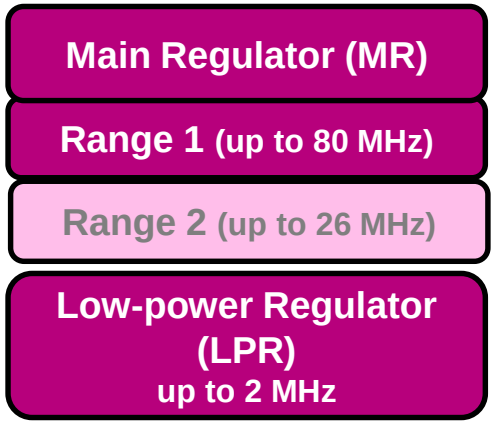
Run mode: Range 1

- All MCU's resources are ON
- System frequency up to the maximum value

Available peripherals

- GPIO
- DMA
- FSMC
- QUADSPI
- BOR
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- IWDG
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC





Range 1 from FLASH
131 µA/MHz at 80 MHz
(10.5 mA)

Available clocks

- HSI
- HSE
- LSI
- LSE
- MSI

Active cell

Clocked-off cell

Cell in power-down

Available Periph and clock

Run mode: Range 2

- Most of MCU's resources are ON
- System frequency is limited

Available peripherals

GPIO
DMA
FSMC
QUADSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC

Available clocks

HSI
HSE
LSI
LSE
MSI

Cortex M4

FLASH memory (1 Mbyte)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Main Regulator (MR)

Range 1 (up to 80 MHz)

Range 2 (up to 26 MHz)

Low-power Regulator (LPR) up to 2 MHz

Range 2 from SRAM1
100 µA/MHz at 26 MHz
(2,6 mA)

Active cell

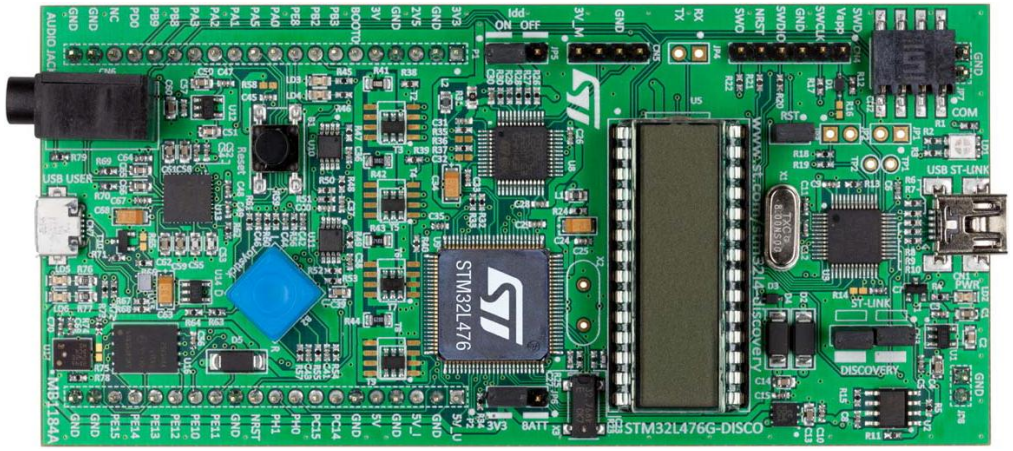
Clocked-off cell

Cell in power-down

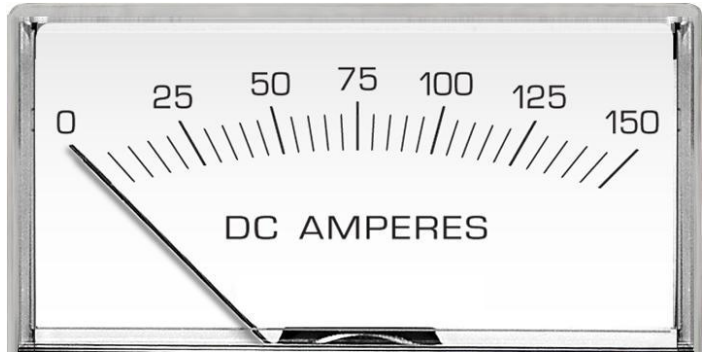
Available Periph and clock

Hands-on: important information

- Examples refer to real applications
- Software for each example was developed using CubeMX code generator for STM32
- Hardware platform for the examples is STM32L476G-DISCO board



- Each example can be examined by attaching ammeter to the board and measuring current

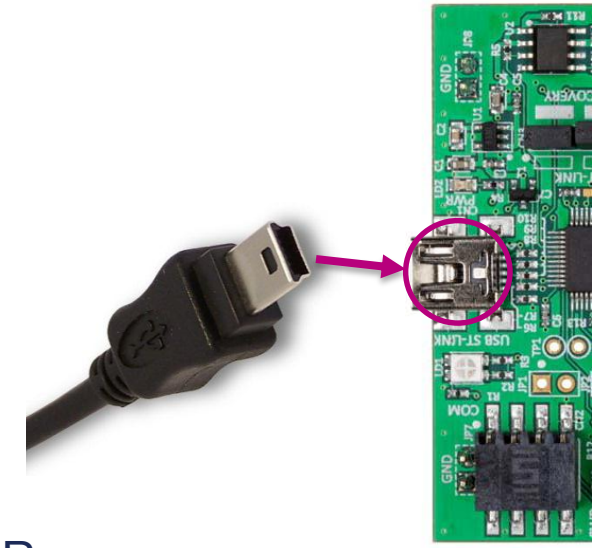


life.augmented

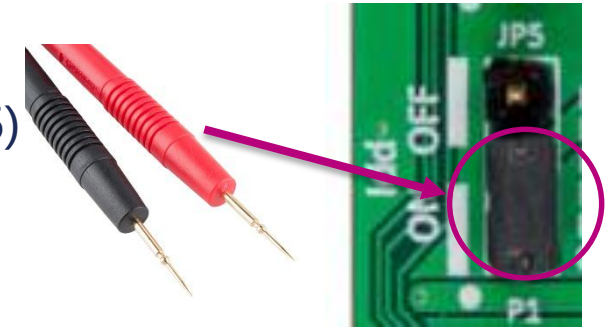
Running hands-on examples

EWARM (IAR toolchain)

- Make sure that software tools are installed
  - IAR EWARM 7.40
  - CubeMX 4.11.0
  - ST-Link driver
- Attach STM32L476G-DISCO board to laptop (mini USB cable required)

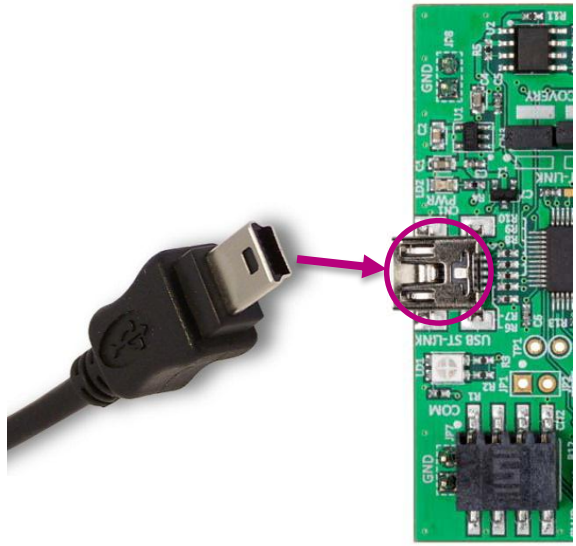


- Find example that you want to examine (they are located on USB stick in "**Hands-on\1_System_Operating_Modes\IAR EWARM**" folder) and open it in IAR EWARM by double-clicking "Project.eww" file
- Run the example by:
  - building it (from Menu: Project->Make or F7)
  - downloading it to MCU (from Menu: Project-> Download->Download Active Application)
  - or debugging it on MCU (from Menu: Project->Download and Debug)
- Measure MCU current consumption by attaching ammeter to Idd connector (JP5) instead of jumper switched to ON position

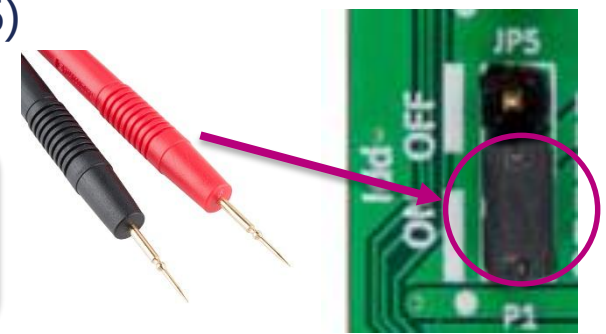


Running hands-on examples
SW4STM32 (AC6 toolchain)

- Make sure that software tools are installed
  - SW4STM32
  - CubeMX 4.11.0
  - ST-Link driver



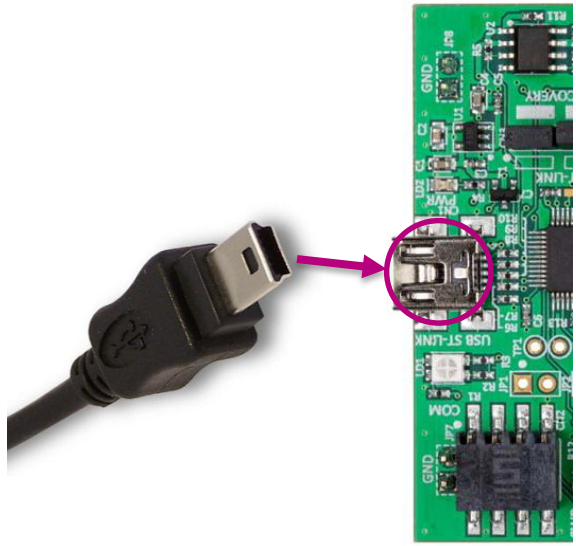
- Attach STM32L476G-DISCO board to laptop (mini USB cable required)
- Create an empty workspace and IMPORT all 7 projects from USB stick (folder **\Hands-on\1_System_Operating_Modes\SW4STM32**)
- Run the example by:
  - Click on its name in projects explorer (to set an active one)
  - building it (from Menu: Project->Build All)
  - debugging it on MCU (from Run->Debug)
- Measure MCU current consumption by attaching ammeter to Idd connector (JP5) instead of jumper switched to ON position



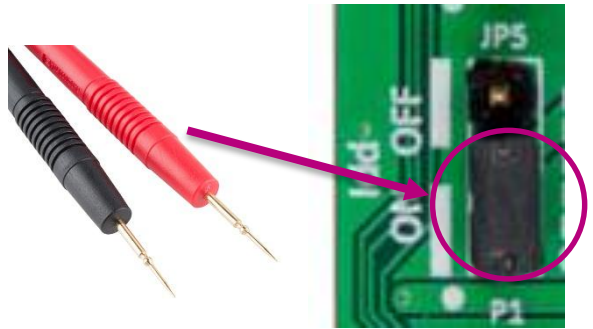
You can refer to dedicated presentation
“SW4STM32 – how to” for more details

Running hands-on examples
using binaries and STLinkUtility

- Make sure that software tools are installed
  - STLinkUtility
  - ST-Link driver
- Attach STM32L476G-DISCO board to laptop (mini USB cable required)



- Find .hex file of the example you want to examine (they are located on USB stick in "**Hands-on\1_System_Operating_Modes\! Binaries**" folder) and open it in STLinkUtility by selecting "File->Open File" menu option.
- Run the example by:
  - Programming the board
  - Reset the board (by black reset button)
- Measure MCU current consumption by attaching ammeter to Idd connector (JP5) instead of jumper switched to ON position

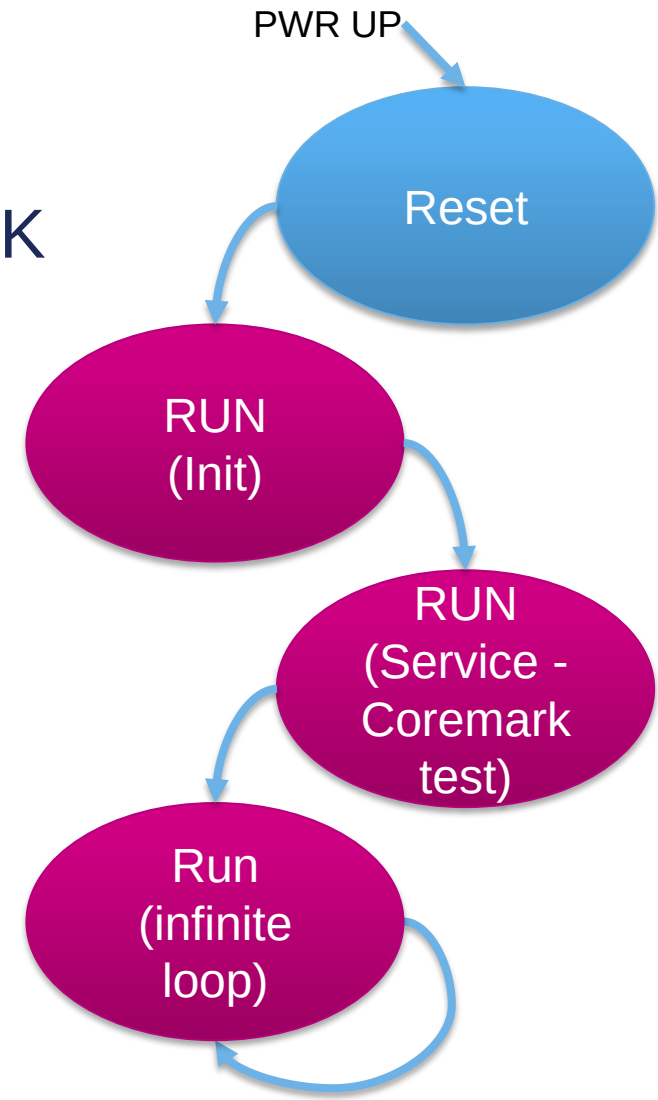


You can refer to dedicated presentation
"STM32 STLink Utility – how to" for more details

life.augmented

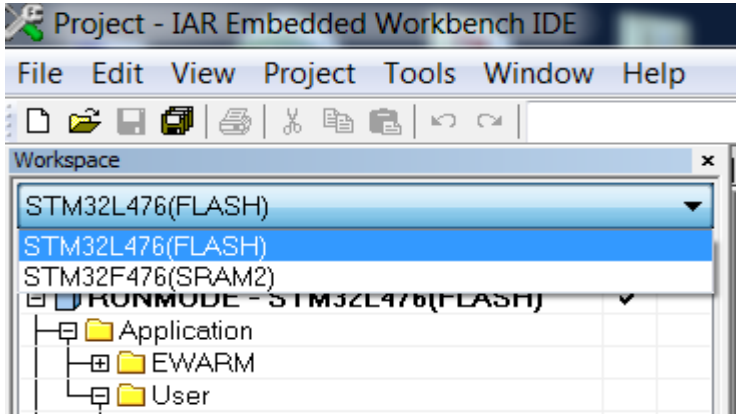
Hands-on example 1: RUN mode

- **Application:** High-performance data processing unit
- **Description of application:** MCU executes COREMARK algorithm
- **Example path:**
  \Hands-on\1_System_Operating_Modes\1_RUNMODE
- **Application's parameters:**
  - Clock: 24MHz (MSI@4MHz) / 80 MHz (MSI@4MHz + PLL),
  - Execution space: FLASH/SRAM memory (binaries for FLASH)



Hands-on: RUN mode configuration

- **Configuration:**
  - Execution space (FLASH or SRAM2) can be changed by selecting a linker file



  - Clock frequency can be changed in ***main.c*** by uncommenting **Clock_MSI24MHZ_Enable()**, PLL 80MHz clock is default

```
94    //ART_Disable();
95    //Clock_MSI24MHZ_Enable();
```

  - Printf instruction is redirected to UART2 / ST-Link Virtual Com Port.
    Jumpers SB13 & SB16 should be soldered. UART parameters: **115200b/s,8,N,1**
  - Virtual Comport Driver should be installed on the target PC

Hands-on: RUN mode results

- **Result can be obtained in Terminal software (like Termite):**

```
Running from address:   0x08004495
Data at address: 0x200001A8
WS:  4, ART: ON
System Clock:  80000000
2K performance run parameters for coremark.
CoreMark Size    : 666
Total ticks      : 303264193
Total time (secs): 3.790802
Iterations/Sec   : 263.796392
Iterations       : 1000
Memory location  : STACK
```

- **Results of COREMARK algorithm (EWARM 7.40):**

Conditions	COREMARK result	Average current consumption
FLASH / CLK=80MHz	263.8	14.3 mA (178 µA/MHz)
SRAM1/SRAM2 CLK=80MHz	269.8	14.1 mA (176 µA/ MHz)
FLASH / CLK=24MHz	80.5	4.5 mA (187 µA/MHz)
SRAM1/SRAM2 CLK=24MHz	80.8	3.24 mA (135 µA/MHz)

Low-power run mode

- Main regulator is OFF
- System frequency is limited

Available peripherals

- GPIO
- DMA
- FSMC
- QUADSPI
- BOR
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- IWDG
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC

Available clocks

- HSI
- HSE
- LSI
- LSE
- MSI

Cortex M4

FLASH memory (1 MB)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Main Regulator (MR)

Range 1 (up to 80 MHz)

Range 2 (up to 26 MHz)

Low-power Regulator (LPR) up to 2 MHz

from FLASH
135 µA/MHz at 2 MHz (269 µA)

From SRAM1
112 µA/MHz at 2 MHz (225 µA)

Active cell

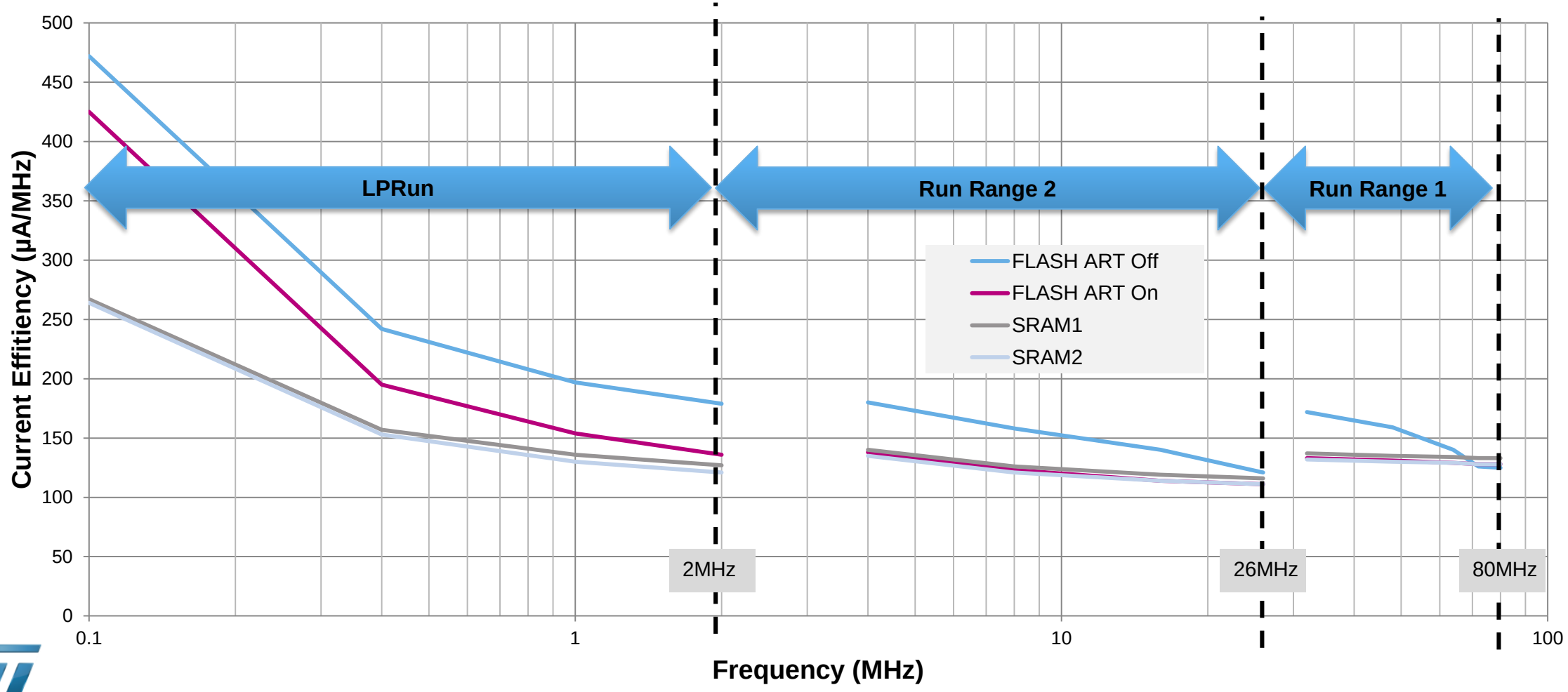
Clocked-off cell

Cell in power-down

Available Periph and clock

Power optimization versus frequency

Flexibility between required performance and consumption

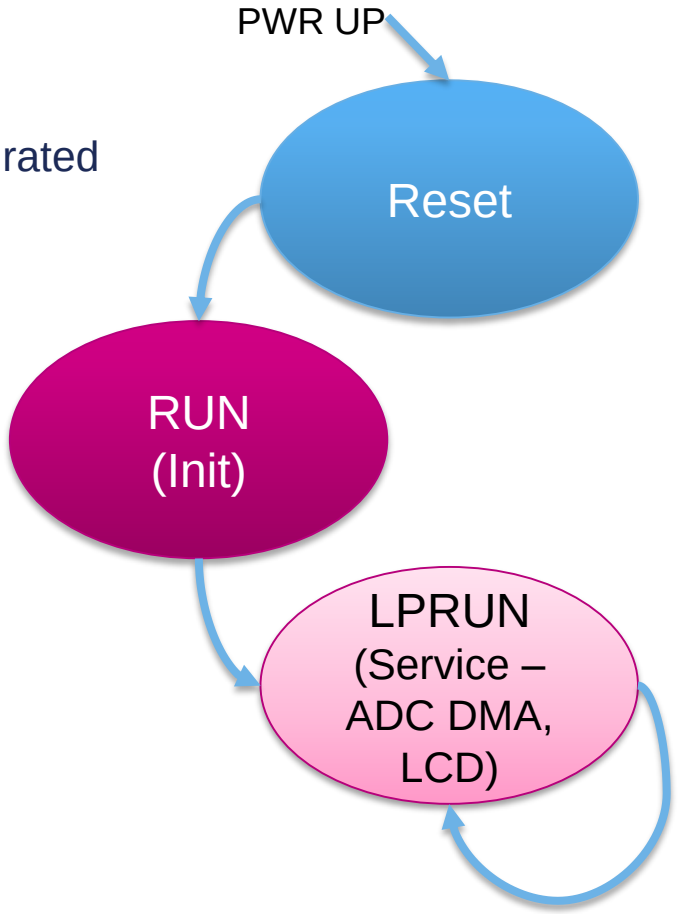


life.augmented

Hands-on example 2: LOW POWER RUN

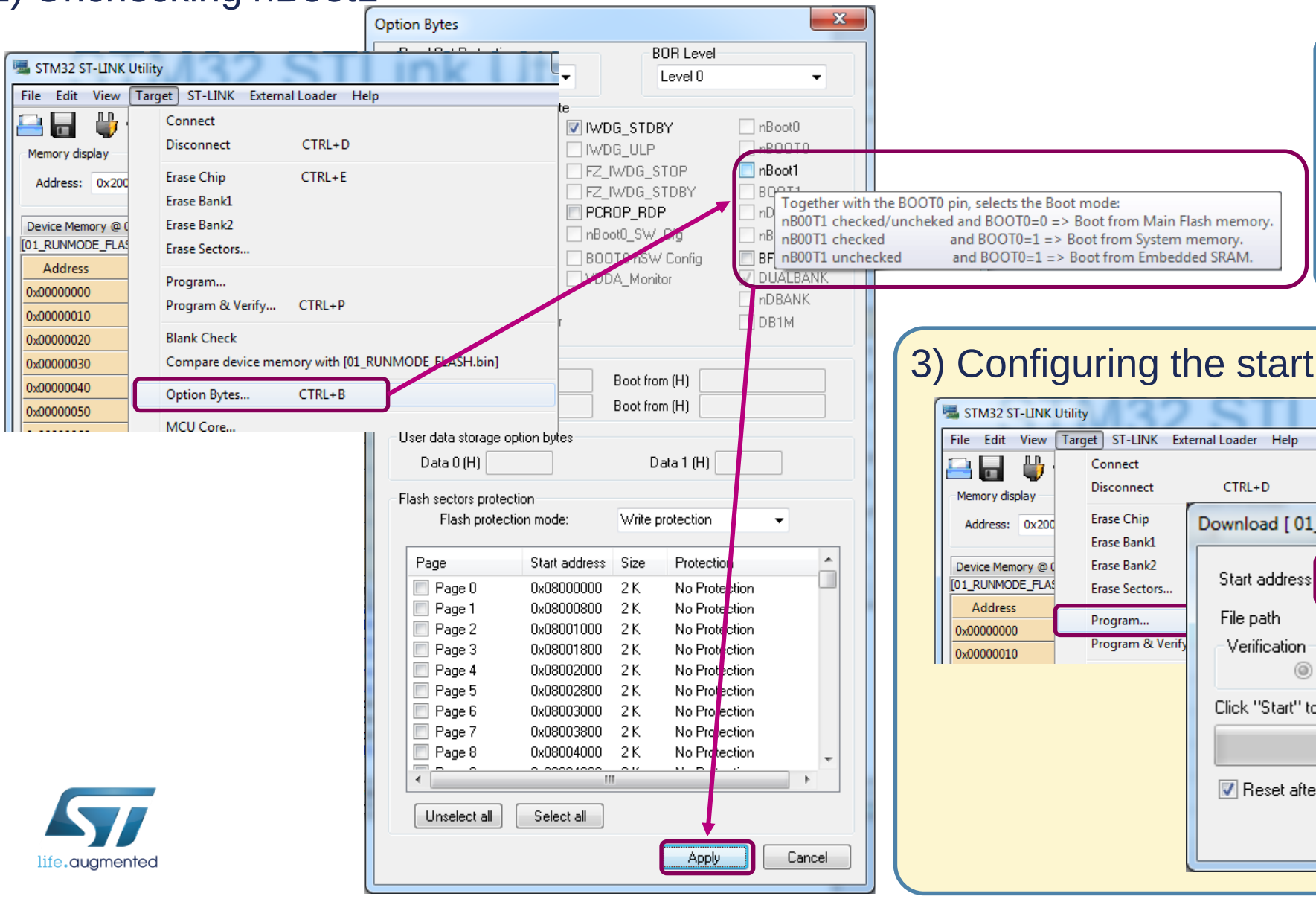
- **Application name:** Fast data acquisition logger (sensor hub), implementation #1
- **Description of application:** every 1ms ADC reads data from temperature sensor integrated in MCU, then data is sent to buffer by DMA, averaged and displayed on LCD.
- **Example path:**

  \Hands-on\1_System_Operating_Modes\2_LOWPOWERRUN_RAM
- **Application's parameters:**
  - Clock: 2 MHz (clock source: MSI),
  - Execution space: **SRAM1 memory**

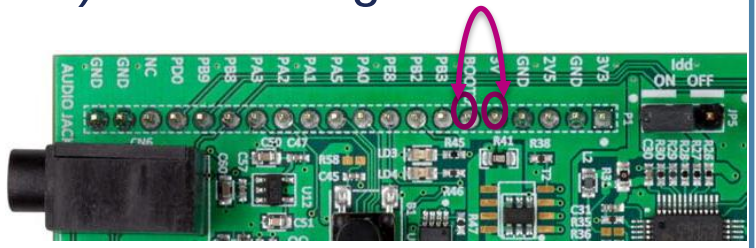


STM32 STLink Utility – loading file to SRAM

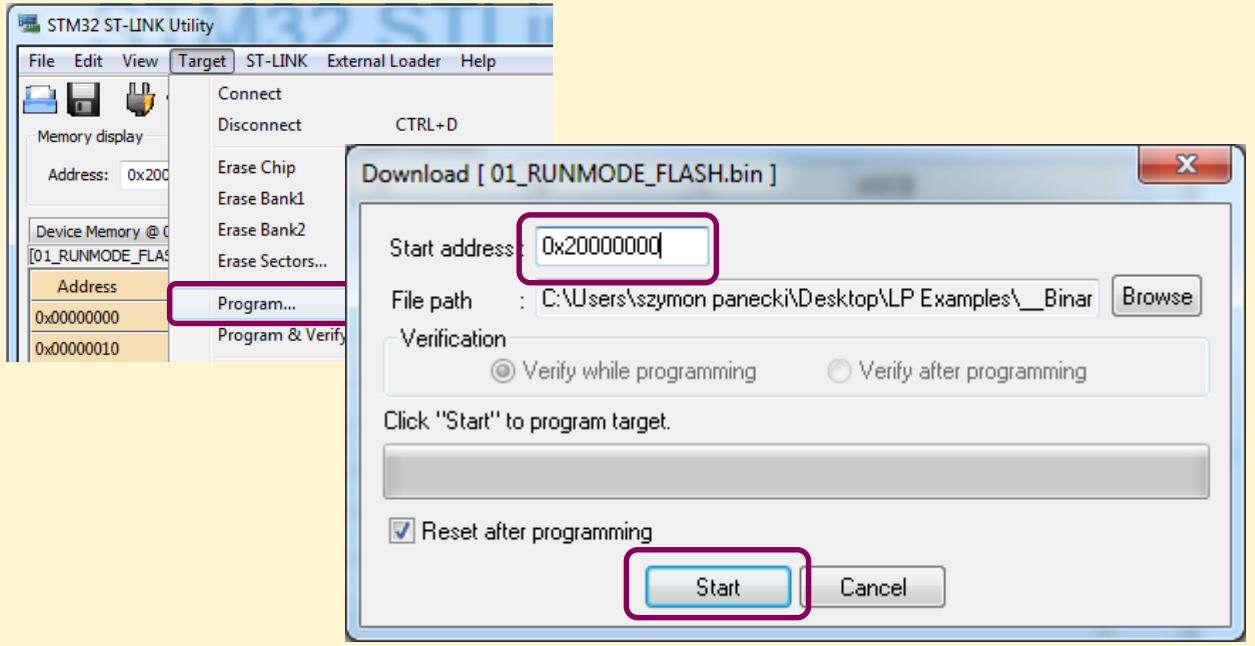
1) Unchecking nBoot1



2) Connecting Boot0 to VDD



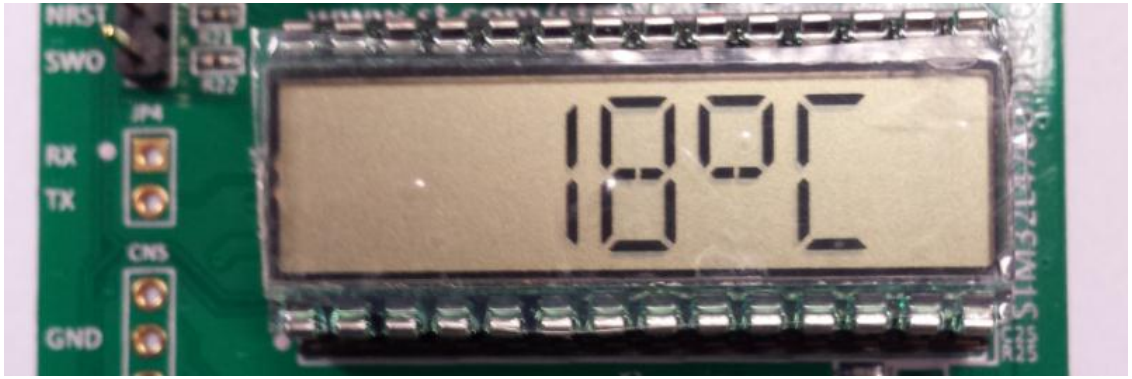
3) Configuring the start address



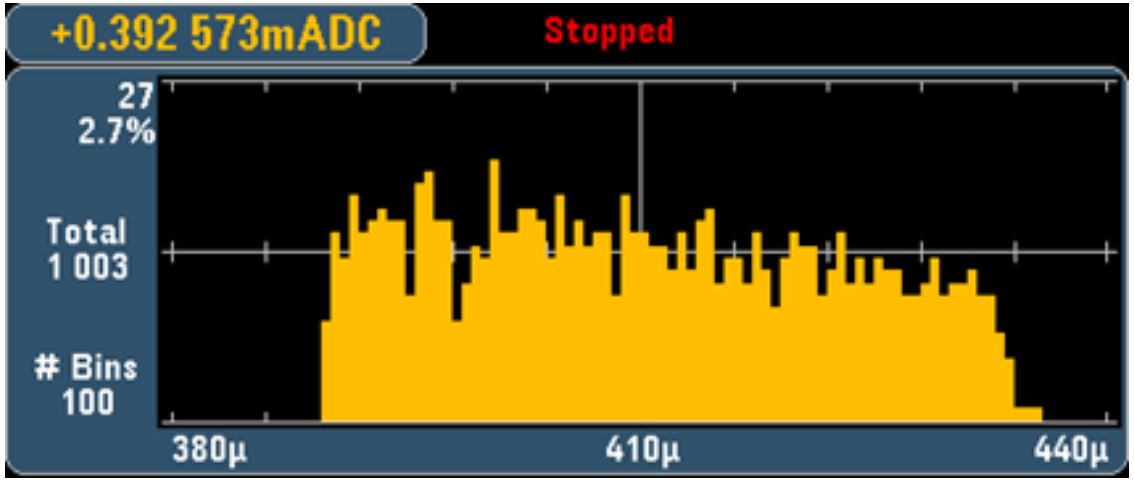
life.augmented

Hands-on example 2: LOW POWER RUN

- **Application behavior:** temperature is displayed on LCD



- **Current consumption:** flat current consumption profile, average current: 410 µA



Sleep mode

- Core is stopped
- All peripherals and clocks are available
- Fastest wakeup time

Wakeup time: 6 cycles

Available peripherals

- GPIO
- DMA
- FSMC
- QUADSPI
- BOR
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- IWDG
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC



Cortex M4

FLASH memory (1 MB)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Main Regulator (MR)

Range 1 (up to 80 MHz)

Range 2 (up to 26 MHz)

Low Power Regulator (LPR) up to 2 MHz

Range 1
37 µA/MHz at 80 MHz (2.96 mA)

Range 2
35 µA/MHz at 26 MHz (0.92 mA)

Available clocks

- HSI
- HSE
- LSI
- LSE
- MSI

Active cell

Clocked-off cell

Cell in power-down

Available Periph and clock

Low-power sleep mode

- Core is stopped
- Main regulator is OFF
- Fastest wakeup time

Wakeup time: 6 cycles

Available peripherals

GPIO
DMA
FSMC
QUADSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC



Cortex M4

FLASH memory (1 MB)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Main Regulator (MR)

Range 1 (up to 80 MHZ)

Range 2 (up to 26 MHZ)

Low Power Regulator (LPR) up to 2 MHz

FLASH ON, SRAMs OFF
48 µA/MHz at 2 MHz
(96 µA)

Available clocks

HSI
HSE
LSI
LSE
MSI

Active cell

Clocked-off cell

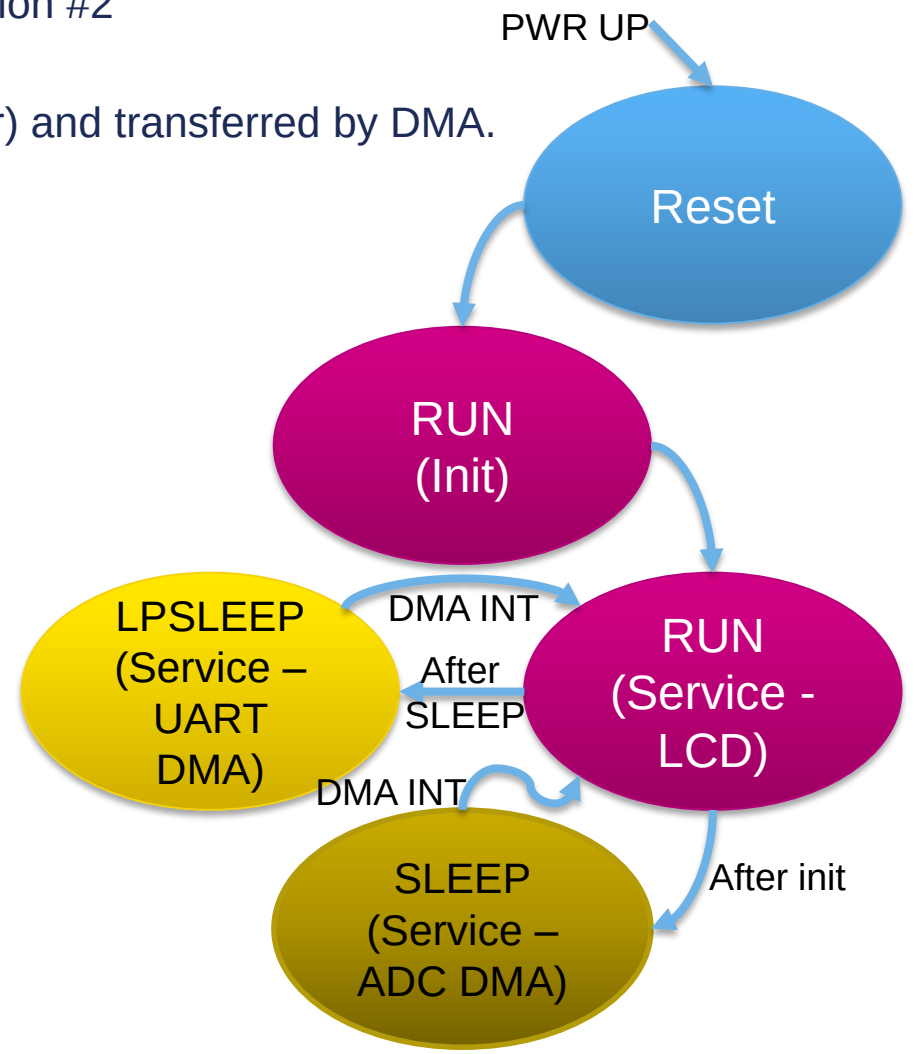
Cell in power-down

Available Periph and clock

Hands-on example 3: SLEEP / LPSLEEP

- **Application name:** Fast data acquisition logger (sensor hub), implementation #2
- **Description of application:**
  In SLEEP mode data is acquired by ADC (from internal temperature sensor) and transferred by DMA.
  In RUN mode data is displayed on LCD.
  In LPSLEEP data is transferred by UART (**9600, 8, N, 1**) with DMA.
- **Example path:**

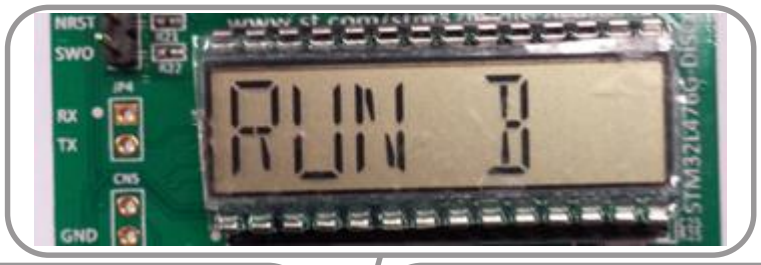
  \Hands-on\1_System_Operating_Modes\3_SLEEP_LPSLEEP
- **Application's parameters:**
  - Clock (clock source: MSI for all):
    4 MHz MSI in SLEEP,
    1 MHz MSI in LPSLEEP,
    4 MHz MSI in RUN,

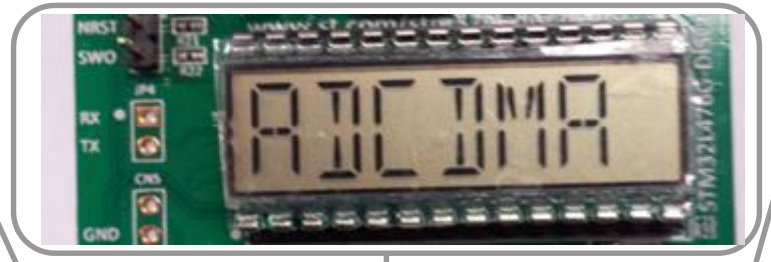


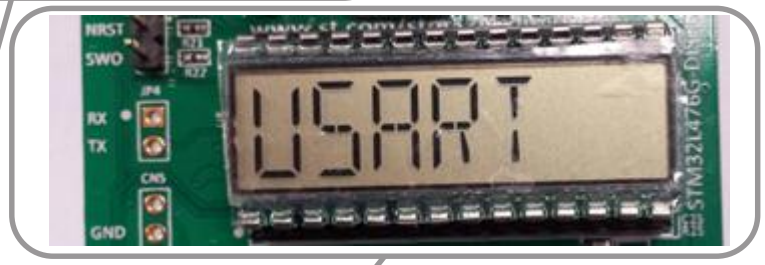
Hands-on example 3: SLEEP / LPSLEEP

- **Application bahvior:**

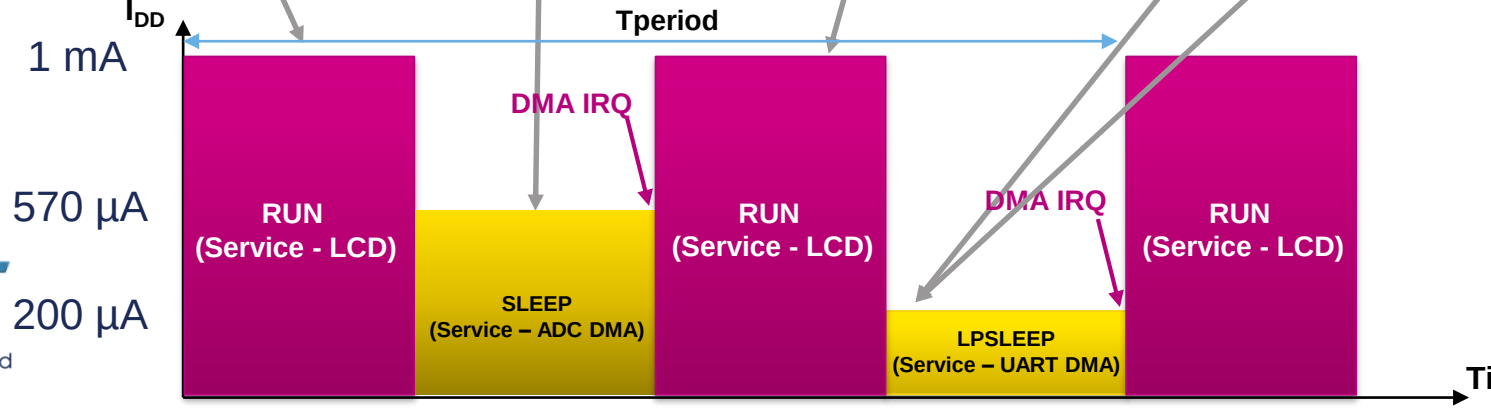


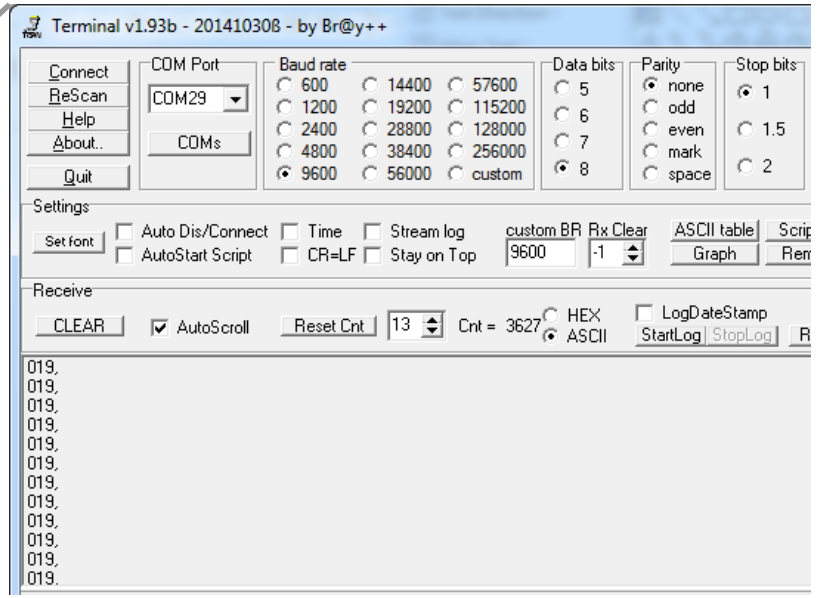






- **Average current consumption:**





life.augmented

Stop modes

Lowest power modes with full retention

- SRAM1, SRAM2 and all peripheral registers retention
- All high-speed clocks are stopped
- LSE (32.768 kHz external oscillator) and LSI (32 kHz internal oscillator) can be enabled
- Several peripherals can stay active to wake device up from Stop mode
- In Stop 2 current consumption is lower; in Stop 1 more active peripherals are supported and wake up time is shorter

life.augmented

Stop 1 mode

Available peripherals

- GPIO
- DMA
- FSMC
- QSPI
- BOR
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- IWDG
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC

I/Os kept, and configurable

w/o RTC: 6.6 µA @ 3.0 V
w/ RTC: 7.1 µA @ 3.0 V

Wakeup time @ 48 MHz:
In SRAM: 4 µs
In FLASH memory: 6 µs



Cortex M4

FLASH memory (1 MB)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Main Regulator (MR)

Low Power Regulator (LPR)

Backup domain

Backup Register (32x32-bits)

RTC

Wake-up event

- NRST
- BOR
- PVD
- PVM
- RTC + Tamper
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SWPMI
- COMP
- LPTIM 1
- LPTIM 2
- IWDG
- GPIOs

Available clocks

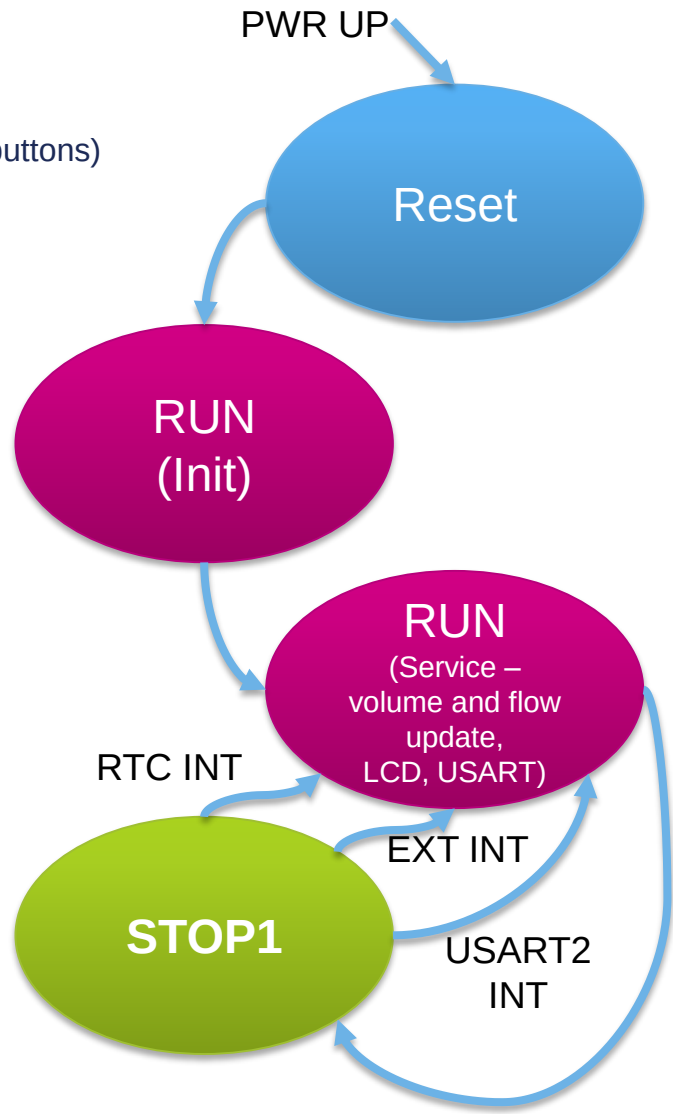
- HSI
- HSE
- LSI
- LSE
- MSI

Active cell | Clocked-off cell | Cell in power-down | Available cell

Hands-on example 4: STOP1

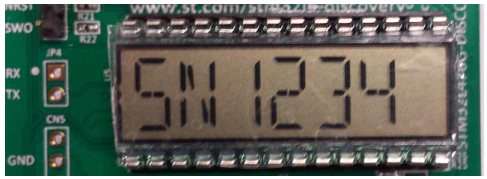
- **Application name:** Water meter, implementation #1 (**with** UART wake-up capability)
- **Description of application:** MCU enters STOP1 mode. Interrupts wake it up.
  - After EXTI interrupt (buttons) MCU provides user interface (displayed message change on Left, Right, Top and Bottom buttons) and counts volume (middle button).
  - After RTC interrupt (once every 60 s), MCU updates average flow (L/m).
  - After USART2 interrupt, MCU sends back volume data by USART2 (**9600/8/1/N**).
- **Example path:**

  \Hands-on\1_System_Operating_Modes\4_WATER_METER_STOP1
- **Application's parameters:**
  - Clock: 16 MHz (clock source: HSI)

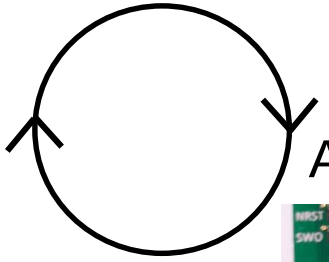


Hands-on example 4: STOP1

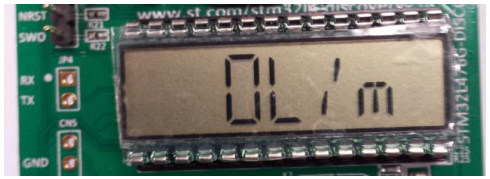
- **Application behavior:**



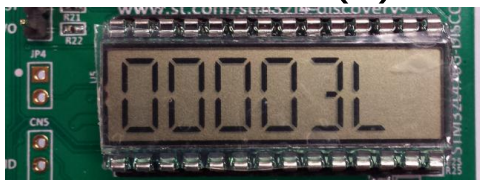
Serial number (SN)



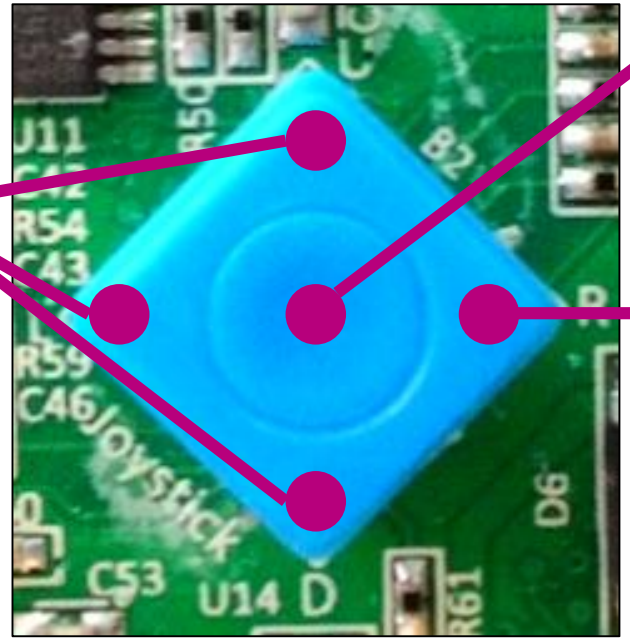
Average flow (L/m)



Total flow (L)



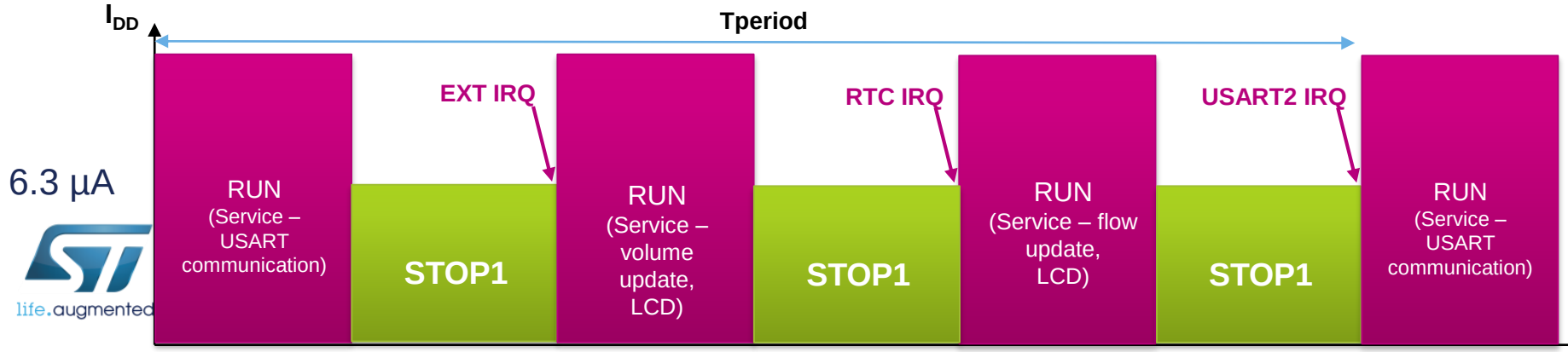
Function change



Pulse counter

Function change

- **Average current consumption:**



6.3 µA

life.augmented

Stop 2 mode

Available peripherals

- GPIO
- DMA
- FSMC
- QSPI
- BOR
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- IWDG
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC

I/Os kept, and configurable

w/o RTC: 1.2 µA @ 3.0 V
w/ RTC: 1.7 µA @ 3.0 V

Wakeup time @ 48 MHz:
In SRAM: 5 µs
In FLASH memory: 8 µs



Cortex M4

FLASH memory (1 MB)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Main Regulator (MR)

Low Power Regulator (LPR)

Backup domain

Backup Register (32x32-bits)

RTC

Wake-up event

- NRST
- BOR
- PVD
- PVM
- RTC + Tamper
- LCD
- LP UART
- I2C 3
- COMP
- LPTIM 1
- IWDG
- GPIOs

Available clocks

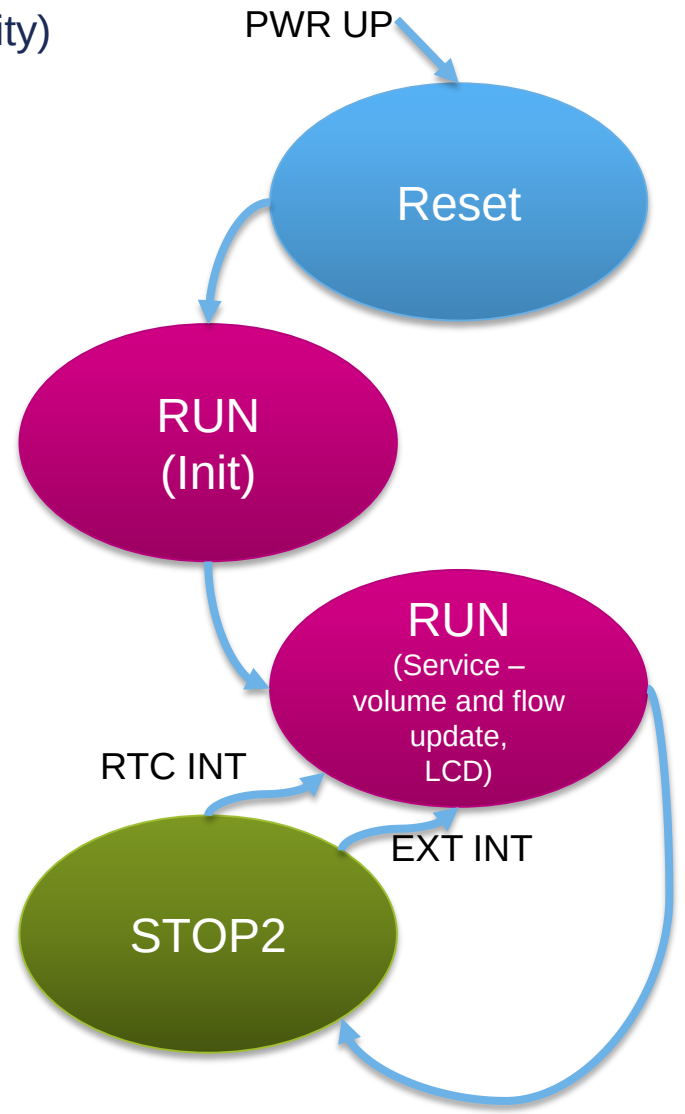
- HSI
- HSE
- LSI
- LSE
- MSI

Active cell | Clocked-off cell | Cell in power-down | Available cell

Hands-on example 5: STOP2

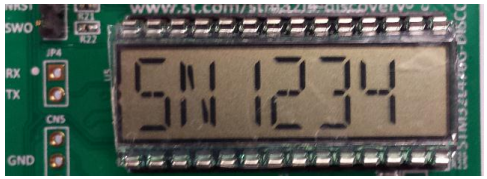
- **Application name:** Water meter, implementation #2 (**without** UART wake-up capability)
- **Description of application:** MCU enters STOP2 mode. Interrupts wake it up.
  After RTC interrupt (every 60 s), MCU updates average flow.
  After EXTI interrupt (buttons) MCU provides user interface and counts volume.
- **Example path:**

  \Hands-on\1_System_Operating_Modes\5_WATER_METER_STOP2
- **Application's parameters:**
  - Clock: 4 MHz (clock source: MSI)
  - LCD can be turned ON/OFF

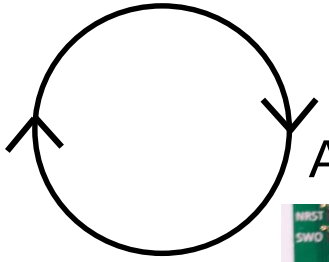


Hands-on example 5: STOP2

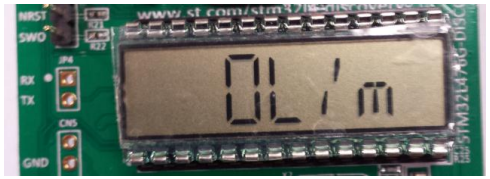
- **Application behavior:**



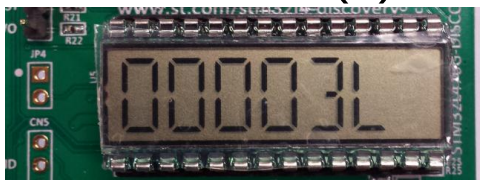
Serial number (SN)



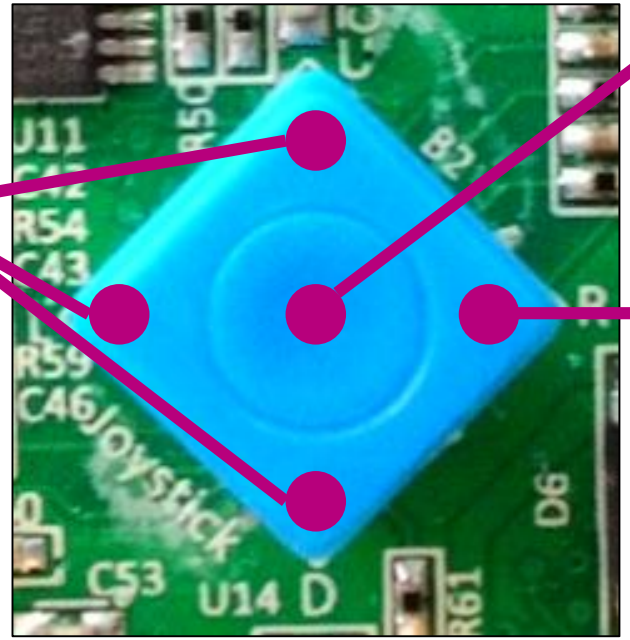
Average flow (L/m)



Total flow (L)



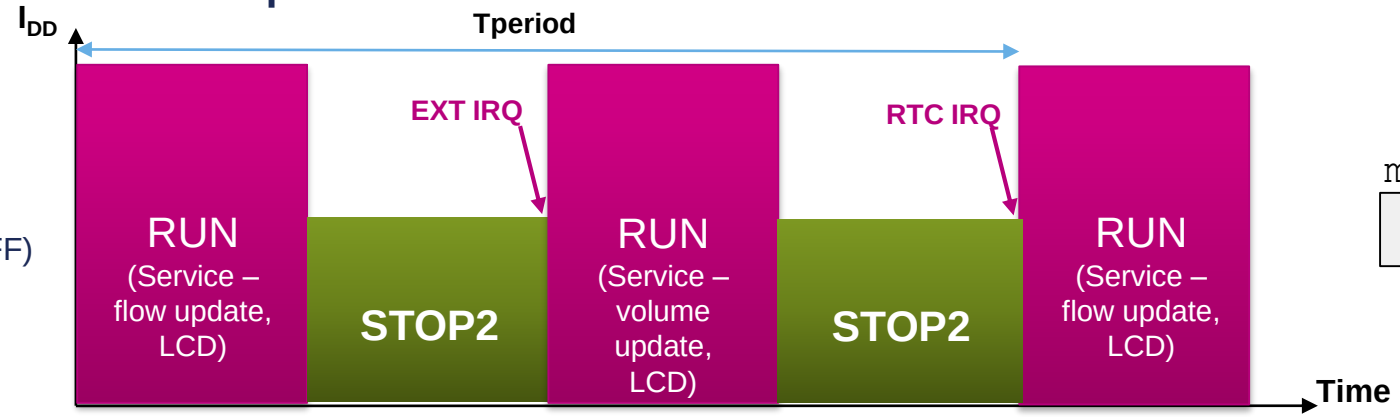
Function change



Pulse counter

Function change

- **Average current consumption:**



2.1 µA
(LCD OFF)

main.c

```
53    //#define LCD_ON
```

life.augmented

Stop 1 & Stop 2 comparison

Voltage range	Stop 1 mode	Stop 2 mode
Consumption	25 °C, 3 V	25 °C, 3 V
	6.6 µA w/o RTC	1.2 µA w/o RTC
Wakeup time to 48 MHz	6 µs in FLASH memory 4 µs in RAM	8 µs in FLASH memory 5 µs in RAM
Wakeup clock	**MSI** (configurable **up to 48 MHz)** or **HSI (**at **16 MHz)**	
Peripherals	LCD, RTC, I/Os, BOR, PVD, PVM, COMPs, IWDG	
	USB (suspend, ADP) 2 LP TIMERs 1 LP UART (Start, address match or byte reception) 5 UARTs (Start, address match or byte reception) 3 I2C (address match) SWPMI (resume from suspend)	1 LP TIMER (**LPTIM1**) 1 LP UART (Start, address match or byte reception) 1 I2C (**I2C3**) (address match)

Lowest power mode with SRAM2 retention

- Ultra low power consumption, down to 115 nA without SRAM retention
- Possibility to **retain 32 kB of SRAM2**
- **5 wakeup pins**: the polarity of each of the **5** wakeup pins is configurable
- Wakeup clock is **MSI configurable from 1 to 8 MHz**

Standby mode with or without SRAM2

Available peripherals

- GPIO
- DMA
- FSMC
- QSPI
- **BOR**
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- **IWDG**
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC

w/ SRAM

w/o RTC: 390 nA @ 3.0 V
w/ RTC: 890 nA @ 3.0 V

w/o SRAM

w/o RTC: 150 nA @ 3.0 V
w/ RTC: 650 nA @ 3.0 V

Wakeup time @ 8 MHz:
In FLASH memory: 14 µs



Cortex M4

Main Regulator (MR)

FLASH memory (1 MB)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Low Power Regulator (LPR)

Wake-up event

- NRST
- BOR
- RTC + Tamper
- IWDG
- 5 WKUP pins

Available clocks

- HSI
- HSE
- LSI
- LSE
- MSI

Backup domain

Backup Register (32x32-bits)

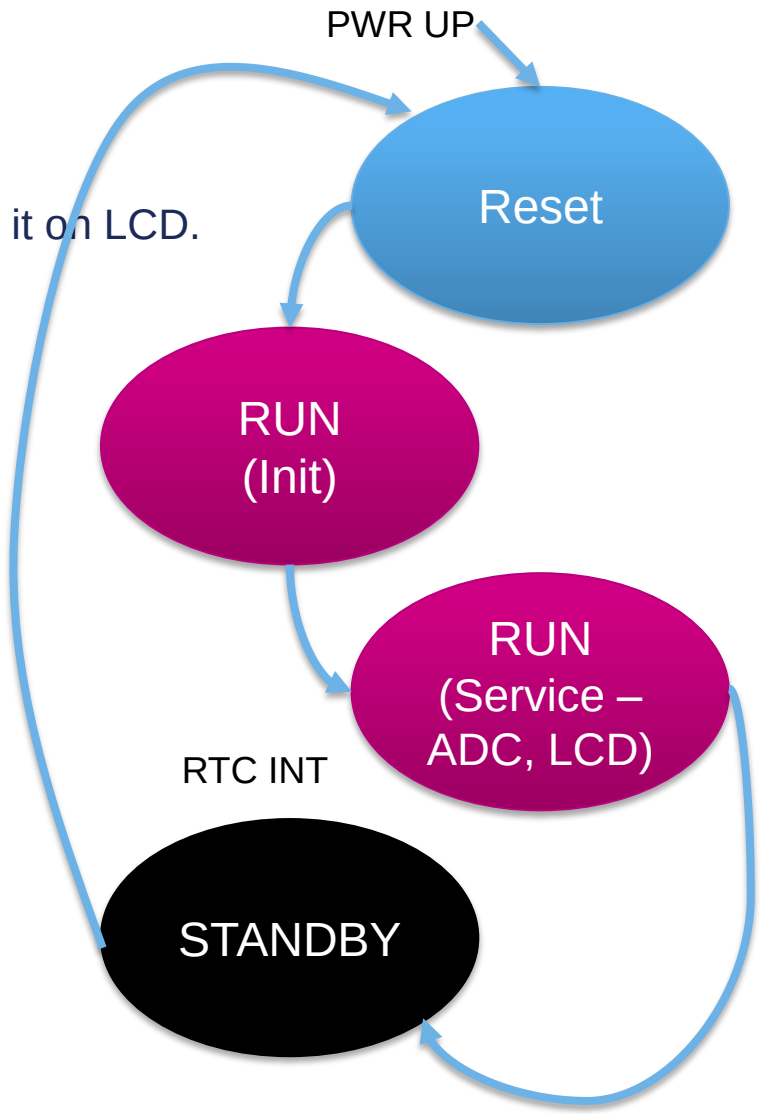
RTC

Active cell | **Clocked-off cell** | **Cell in power-down** | **Available cell**

Hands-on example 6: STANDBY

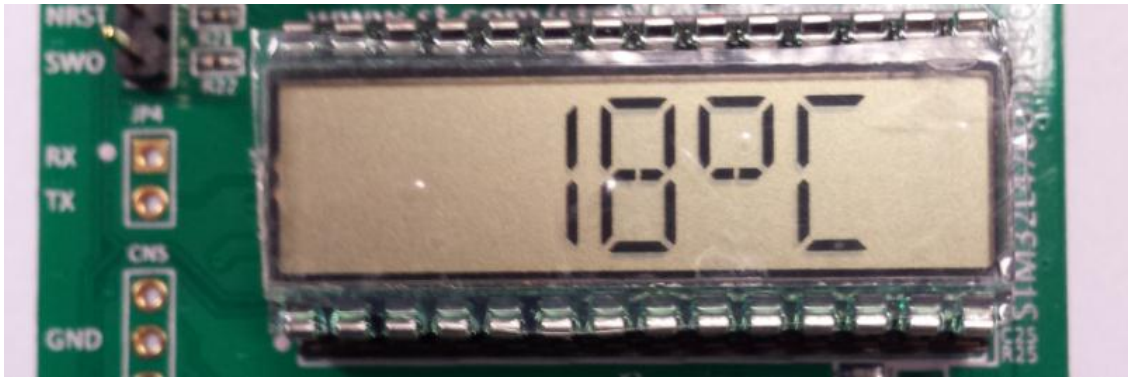
- **Application name:** Long term temperature logger
- **Description of application:** MCU enters STANDBY mode.
  After each RTC wake up, MCU reads ADC data (internal temperature sensor) and displays it on LCD.
- **Example path:**

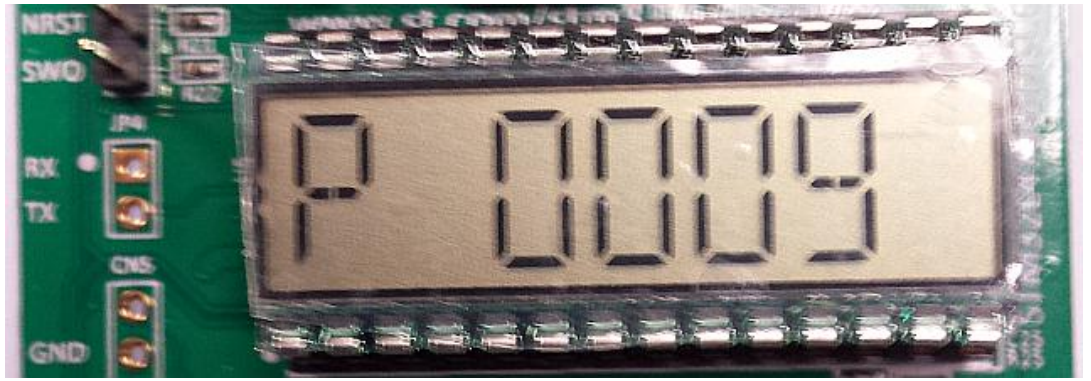
  \Hands-on\1_System_Operating_Modes\6_LONGTERM_LOGGER_STANDBY_SRAM2
- **Application's parameters:**
  - Clock: 4 MHz (clock source: MSI)



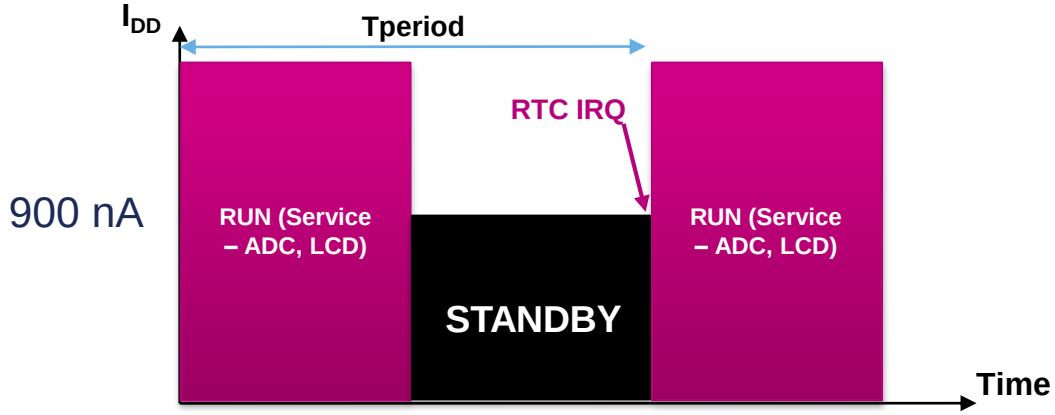
Hands-on example 6: STANDBY

- **Application behavior:** each 10 seconds MCU wakes up, performs task and backs to sleep





- **Average current consumption:**



life.augmented

Shutdown mode

Lowest power mode

- Similar to Standby but
  - **NO power monitoring:** no BOR, no switch to VBAT
  - **NO LSI**, no IWDG
- 128-byte **backup registers**
- Wakeup sources: **5 wakeup pins**, **RTC**
- Wakeup clock is **MSI 4 MHz**

life.augmented

Shutdown mode

Available peripherals

- GPIO
- DMA
- FSMC
- QSPI
- BOR
- PVD, PVM
- LCD
- USB OTG
- USART
- LP UART
- I2C 1 / I2C 2
- I2C 3
- SPI
- CAN
- SDMMC
- SWPMI
- SAI
- DFSDM
- ADC
- DAC
- OPAMP
- COMP
- Temp Sensor
- Timers
- LPTIM 1
- LPTIM 2
- IWDG
- WWDG
- Systick Timer
- Touch Sens
- RNG
- AES
- CRC

I/Os can be configured
w/ or w/o pull-up
w/ or w/o pull-down
But floating when exit from Shutdown

w/o RTC: 60 nA @ 3.0 V
w/ RTC: 550 nA @ 3.0 V

Wakeup time @ 4 MHz:
In FLASH memory: 250 µs



Cortex M4

Main Regulator (MR)

FLASH memory (1 Mbyte)

SRAM 1 (96 kB)

SRAM 2 (32 kB)

Low Power Regulator (LPR)

Wake-up event

- NRST
- RTC + Tamper
- 5 WKUP pins

Available clocks

- HSI
- HSE
- LSI
- LSE
- MSI

Backup domain

Backup Register (32x32-bits)

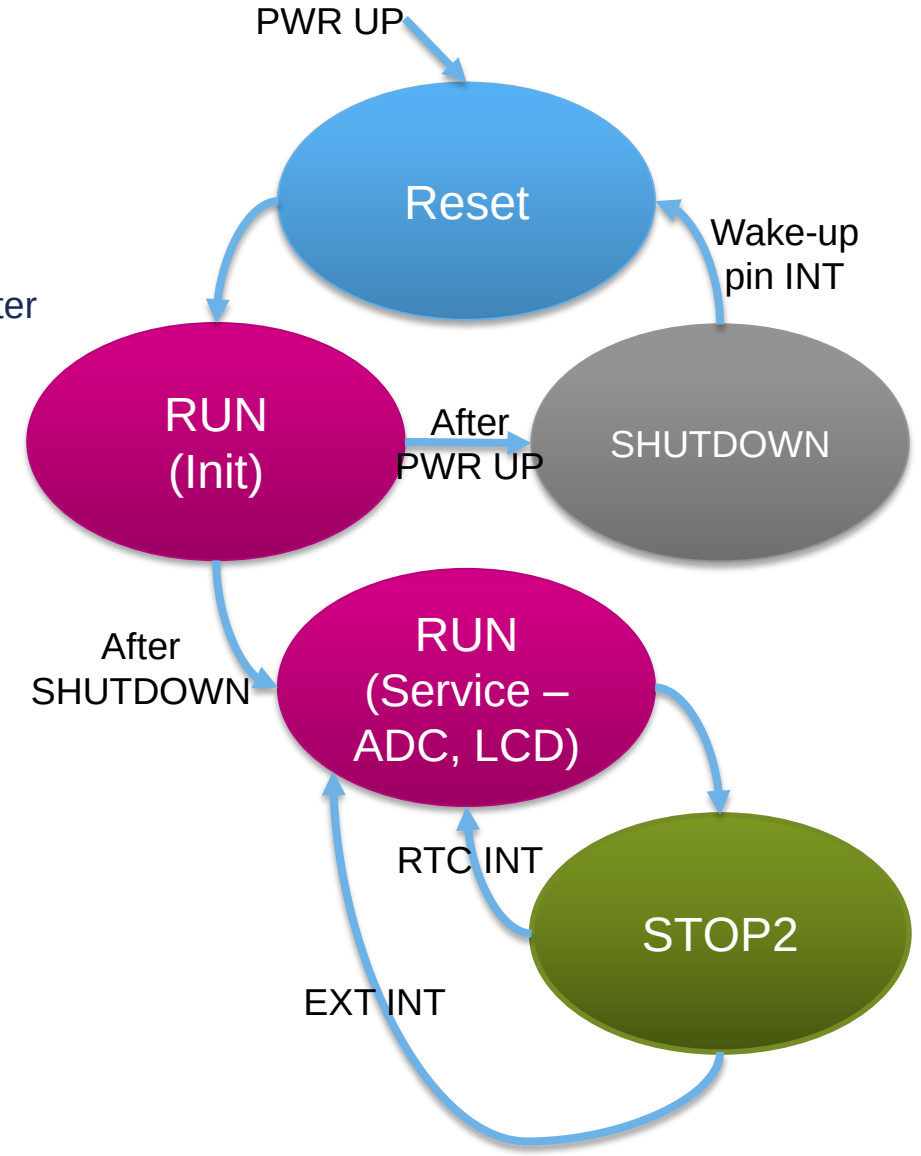
RTC

Active cell | Clocked-off cell | Cell in power-down | Available cell

Hands-on example 7: SHUTDOWN

- **Application name:** Water meter, implementation #3 (with SHUTDOWN mode).
- **Description of application:**
  MCU enters SHUTDOWN mode. This refers to behavior, when already configured battery-operated device is held in warehouse and current consumption should be as low as possible. When the center button is pressed and held (**for 2 seconds**), MCU starts to execute water meter application (**example 5**). This refers to behavior after device is delivered to customer.
- **Example path:** … :

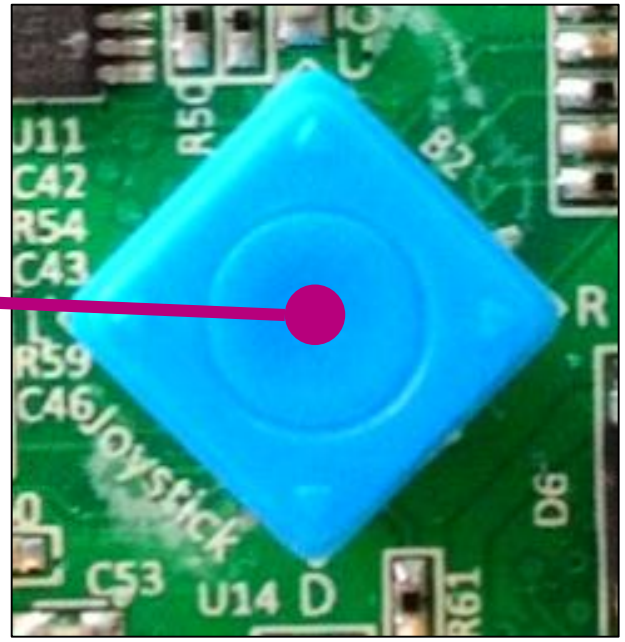
  \Hands-on\1_System_Operating_Modes\7_WATER_METER SHUTDOWN_STOP2
- **Application's parameters:**
  - Clock: 4 MHz (clock source: MSI)



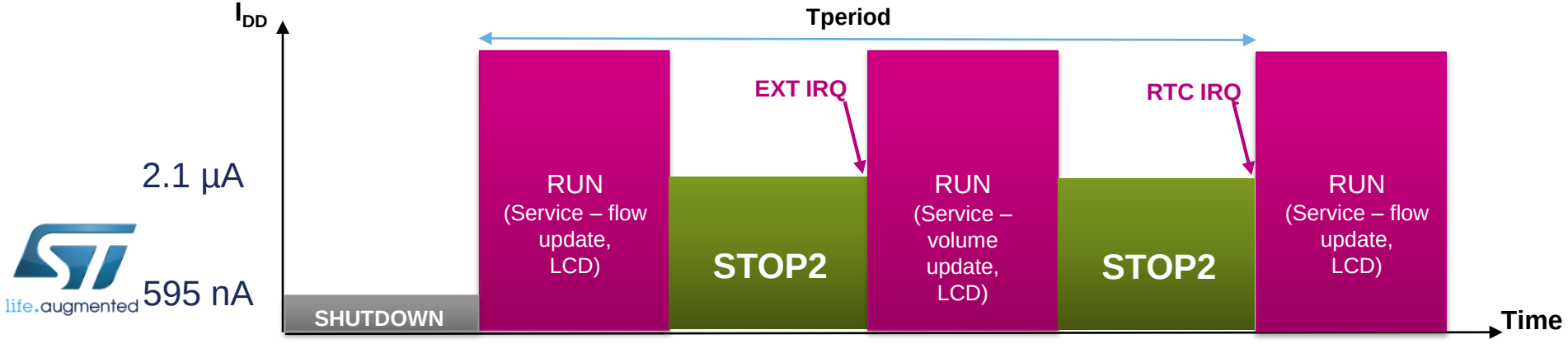
Hands-on example 7: SHUTDOWN

- **Application behavior:**

MCU is in Shutdown mode. To wake it up press center joystick button. Then water meter application (with STOP2 mode) will bee executed.



- **Average current consumption:**



How to associate LP modes

- **RUN:** do you need high computation power, but would like to be green?
- **LOW POWER RUN:** do you have weak power source, but supercapacitors are too costly?
- **SLEEP/LPSLEEP:** do you need peripherals running all the time, but power budget is limited?
- **STOP:** do you need balance between wake up time and low power consumption?
- **STANDBY:** do you need to consume very little power, but still SRAM needs to be retained?
- **SHUTDOWN:** is a low power consumption a top priority in your design?







The End



www.st.com/stm32l4

life.augmented



CubeMX low power lab

life.augmented

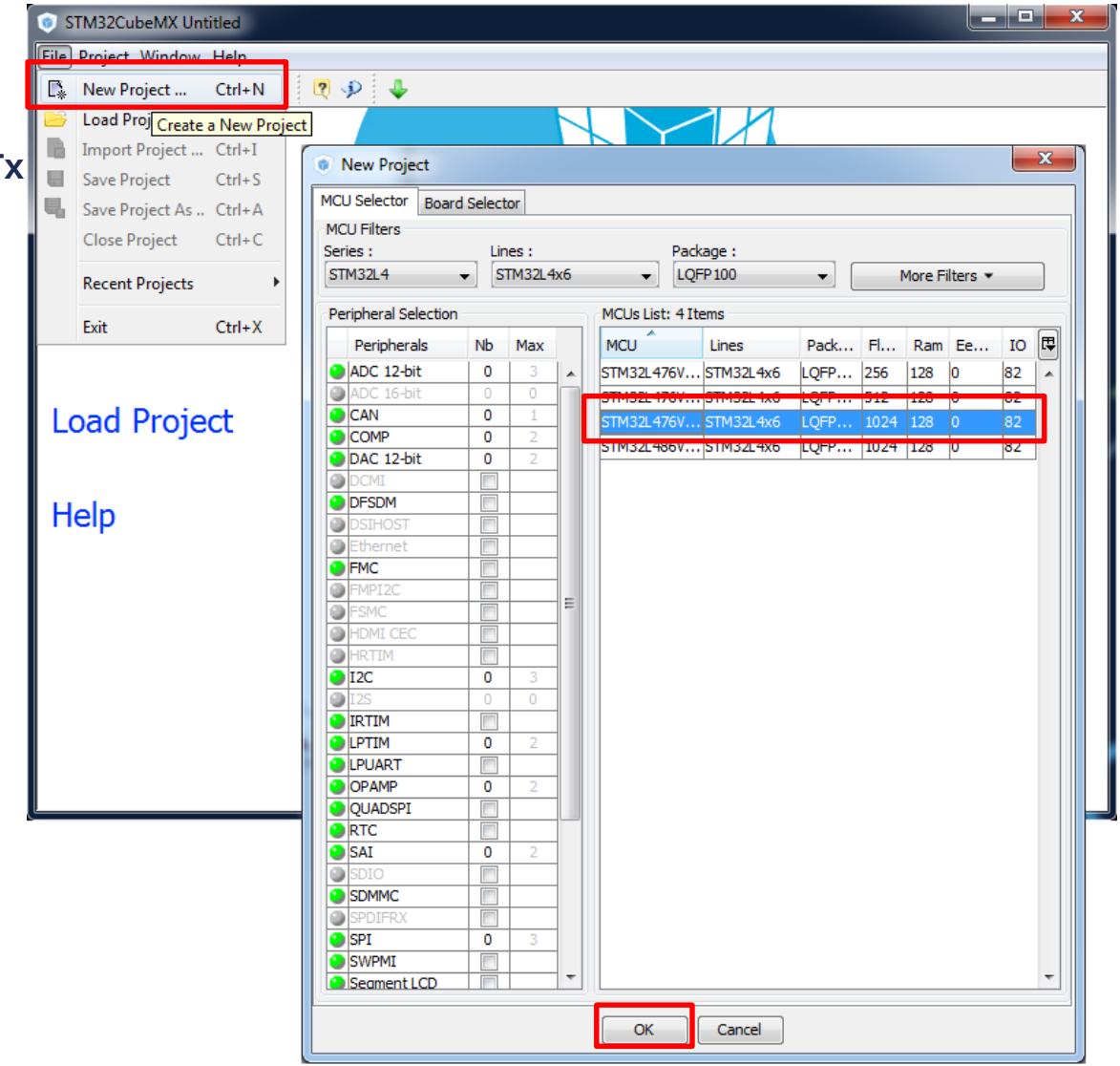
Low power application

- Goal
  - To demonstrate how easy is to create low power application from scratch (example application behavior: MCU leaves STOP2 mode after button press, changes LED state and returns to STOP2 mode)
  - To verify the current consumption by measuring it with ammeter
- Objective
  - To become familiar with CubeMX by configuring MCU peripherals (GPIO, EXTI, NVIC) with GUI and generating source code for IDE
  - To learn how to enter and leave STOP2 mode with HAL
  - To learn how to control GPIOs

Low power application

- Create project in CubeMX
  - To create new project select from Menu: **File > New Project**
  - Select filters to shrink MCU list:

    STM32L4 -> STM32L4x6 -> LQFP100 package -> STM32L476VGTx
  - Press **OK** to accept

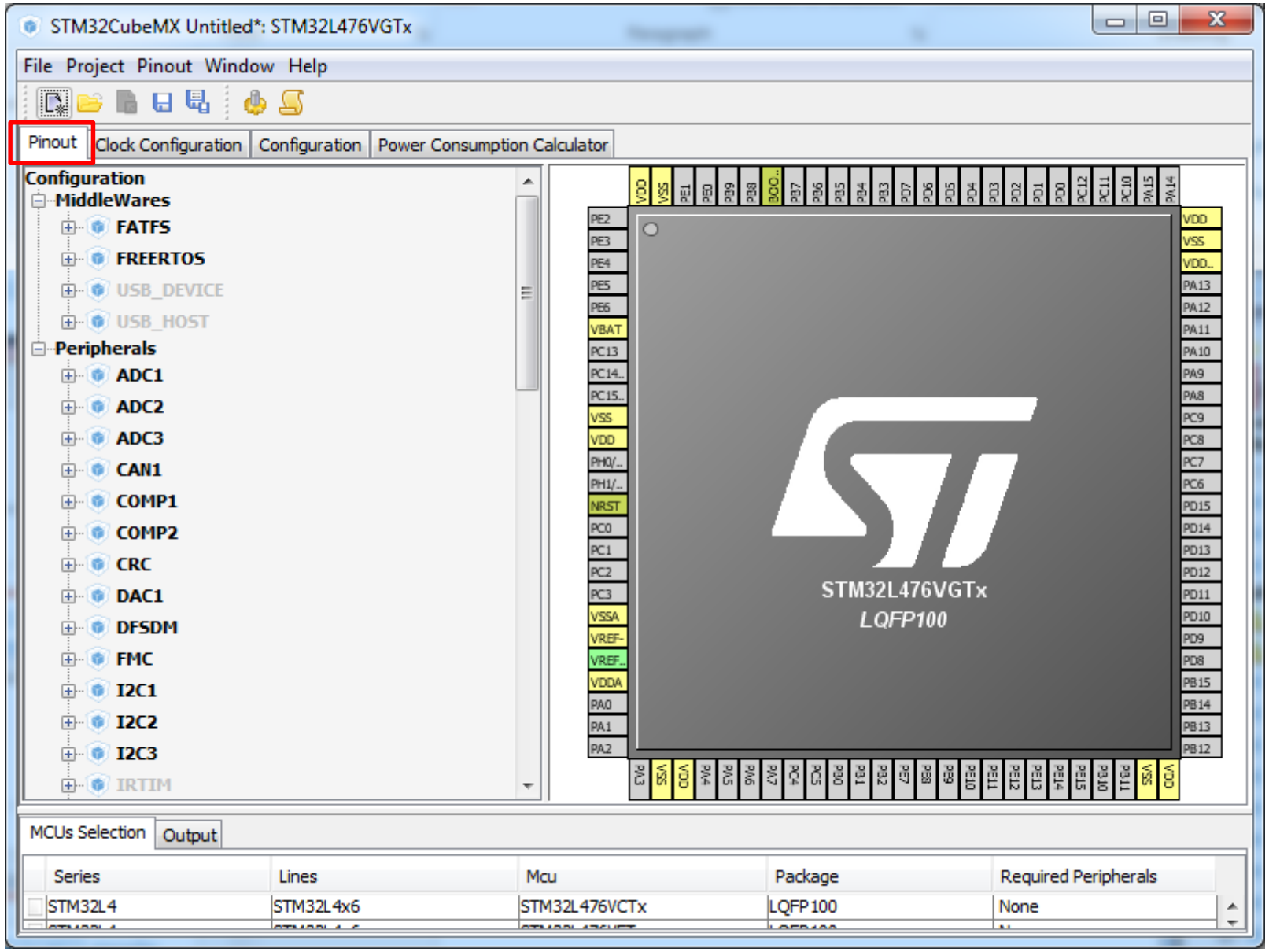


life.augmented

Low power application

- Configure GPIOs
  - Open **Pinout** tab
  - Configure PA0 and PE8

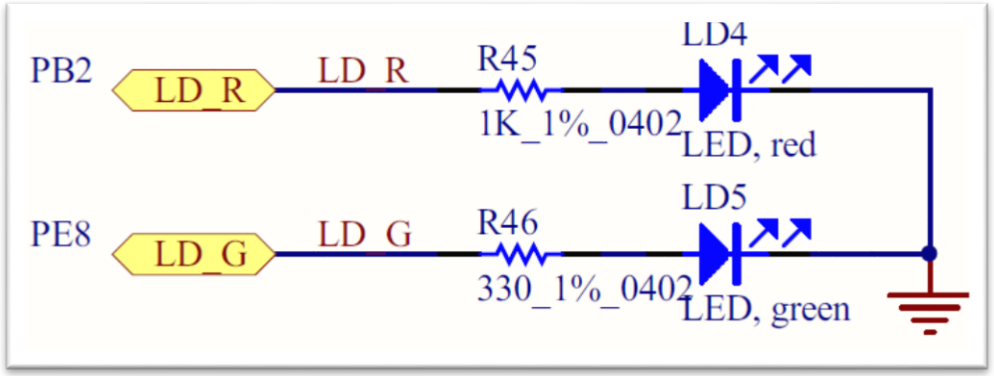
  (details on next slides)

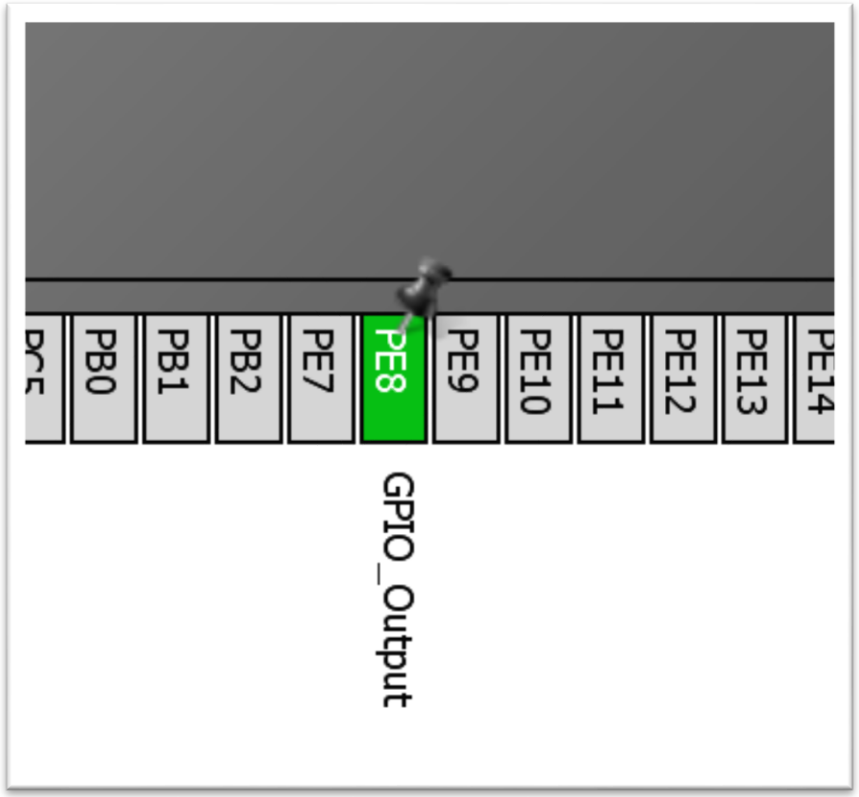


life.augmented

Low power application

- Configure LED (PE8) pin
  - Click on a **PE8** pin in a **Pinout** tab
  - From the menu, which will open, select **GPIO_Output**

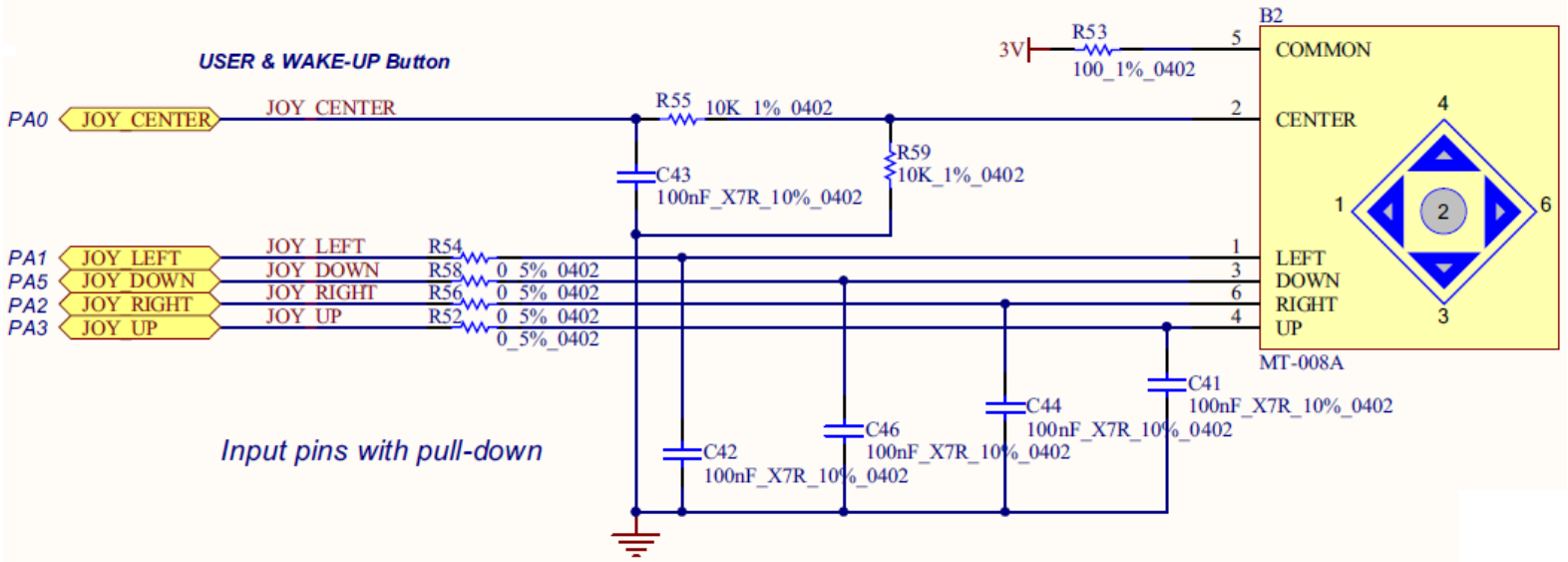


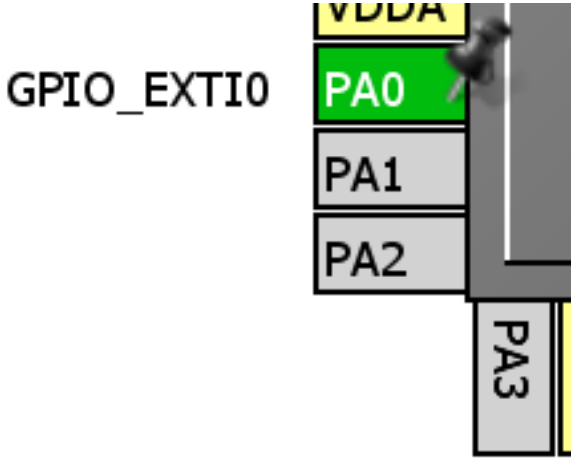


life.augmented

Low power application

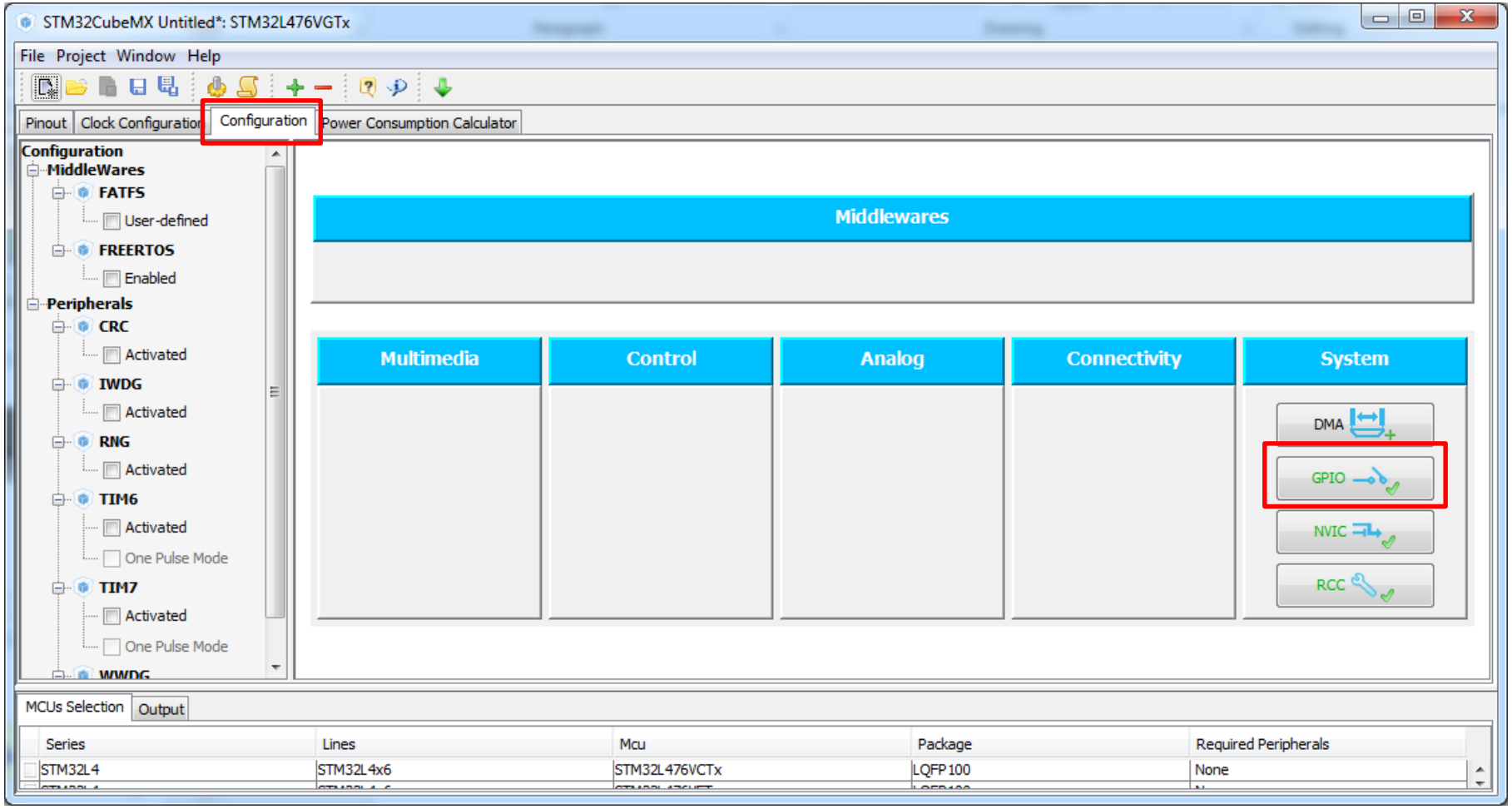
- Configure Button (PA0) pin
  - Click on a **PA0** pin in a **Pinout** tab
  - From the menu, which will open, select **GPIO_EXTI0**





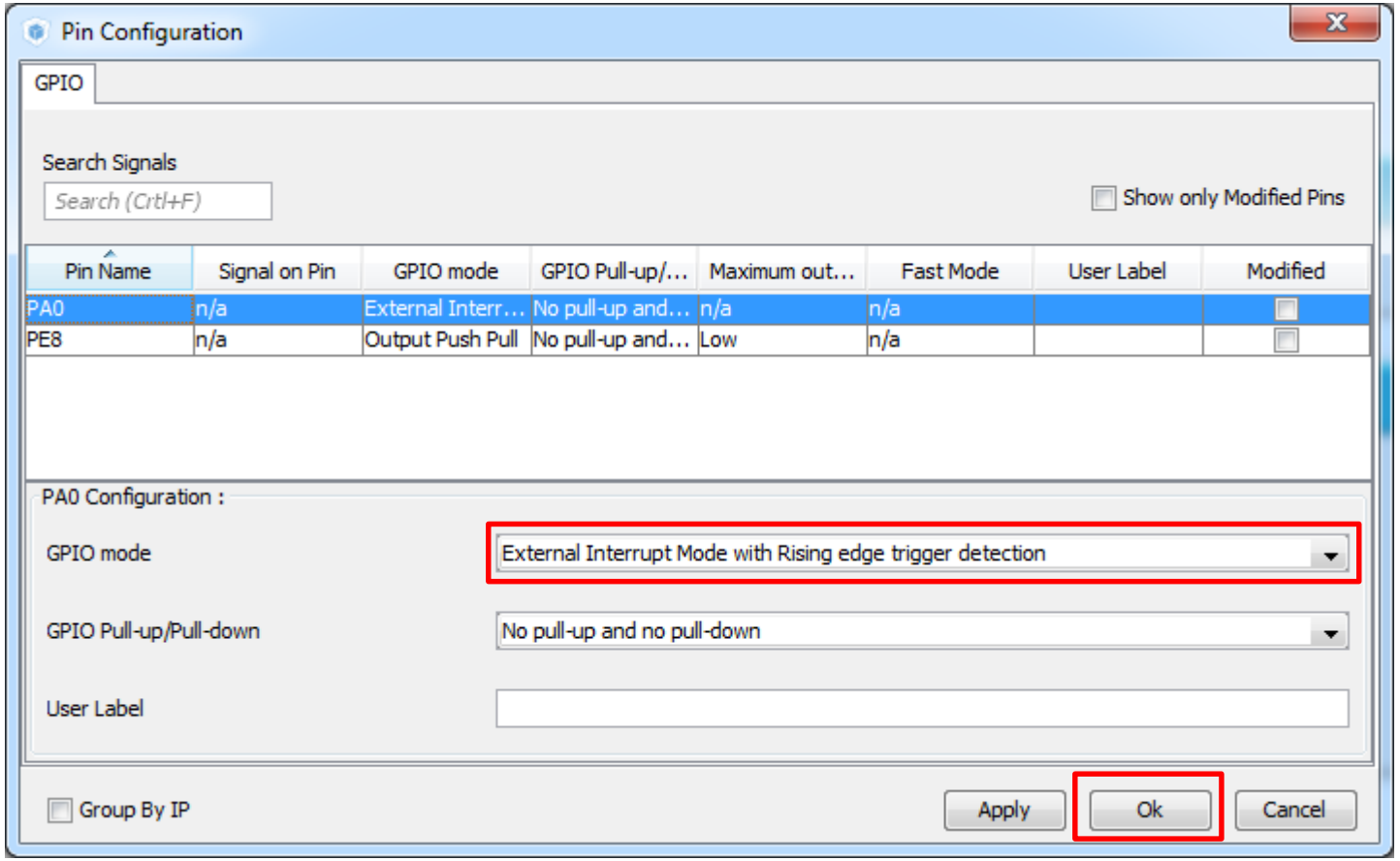
Low power application

- GPIOs configuration
  - Click on a **GPIO** button in **Configuration** tab



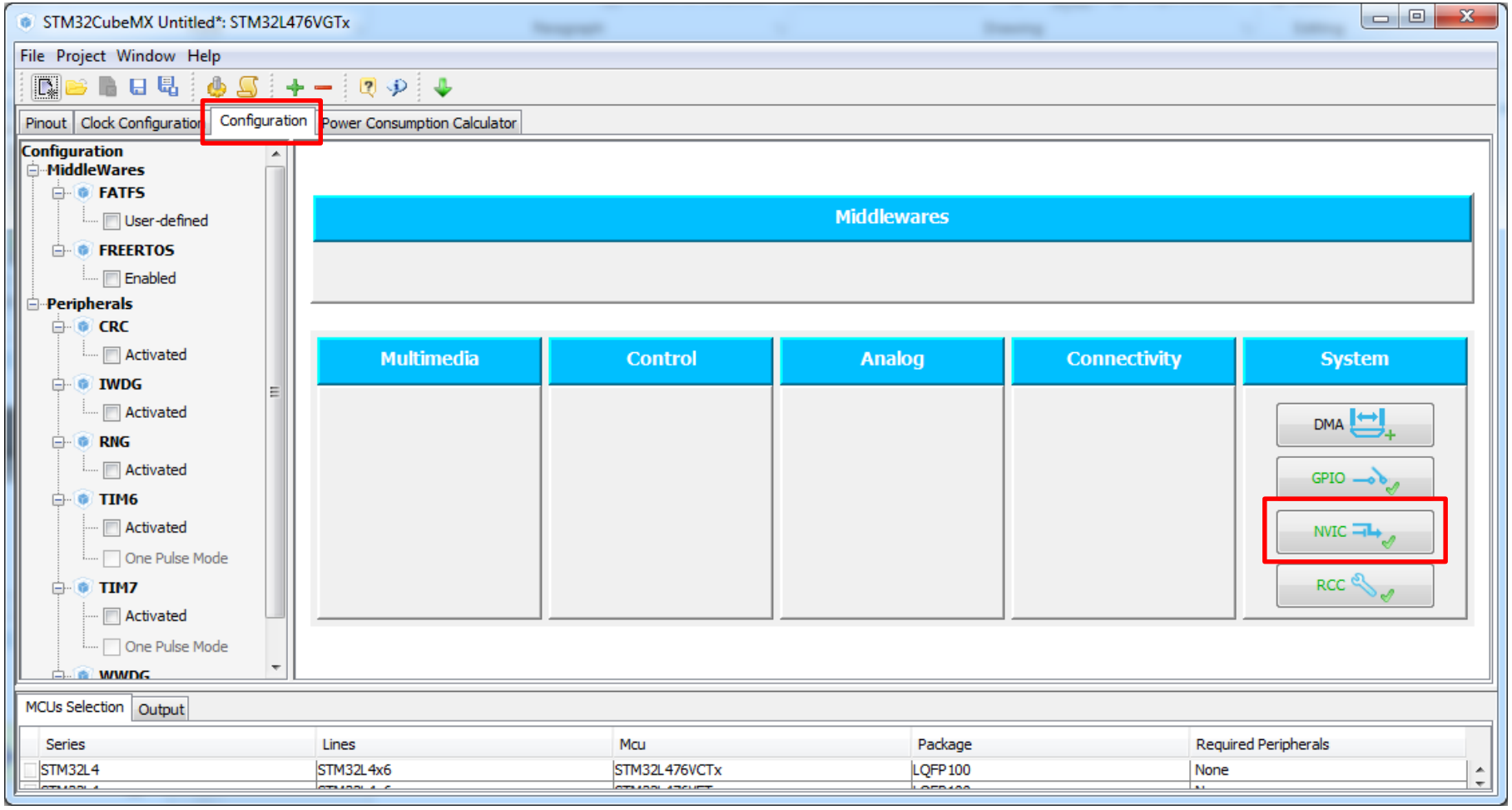
Low power application

- GPIO (Pins) configuration
  - Default configuration for PA0 and PE8 is OK.
  - Just make sure that PA0 GPIO mode is **External Interrupt Mode with Rising edge trigger** detection
  - Press **OK** button to accept



Low power application

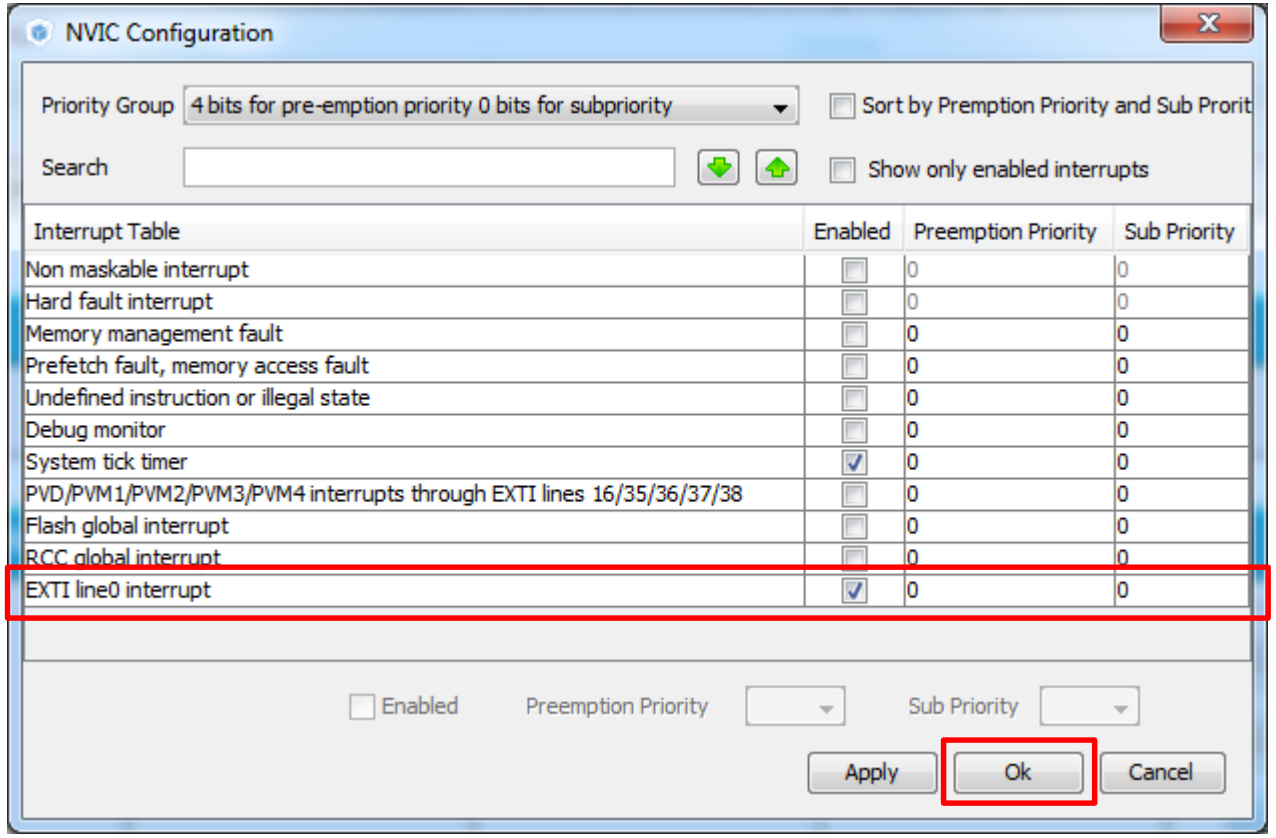
- NVIC (interrupts) configuration
  - Click on **NVIC** button in **Configuration** tab



life.augmented

Low power application

- NVIC (interrupts) configuration
  - Enable interrupt for EXTI Line0
  - Click **OK** button to accept

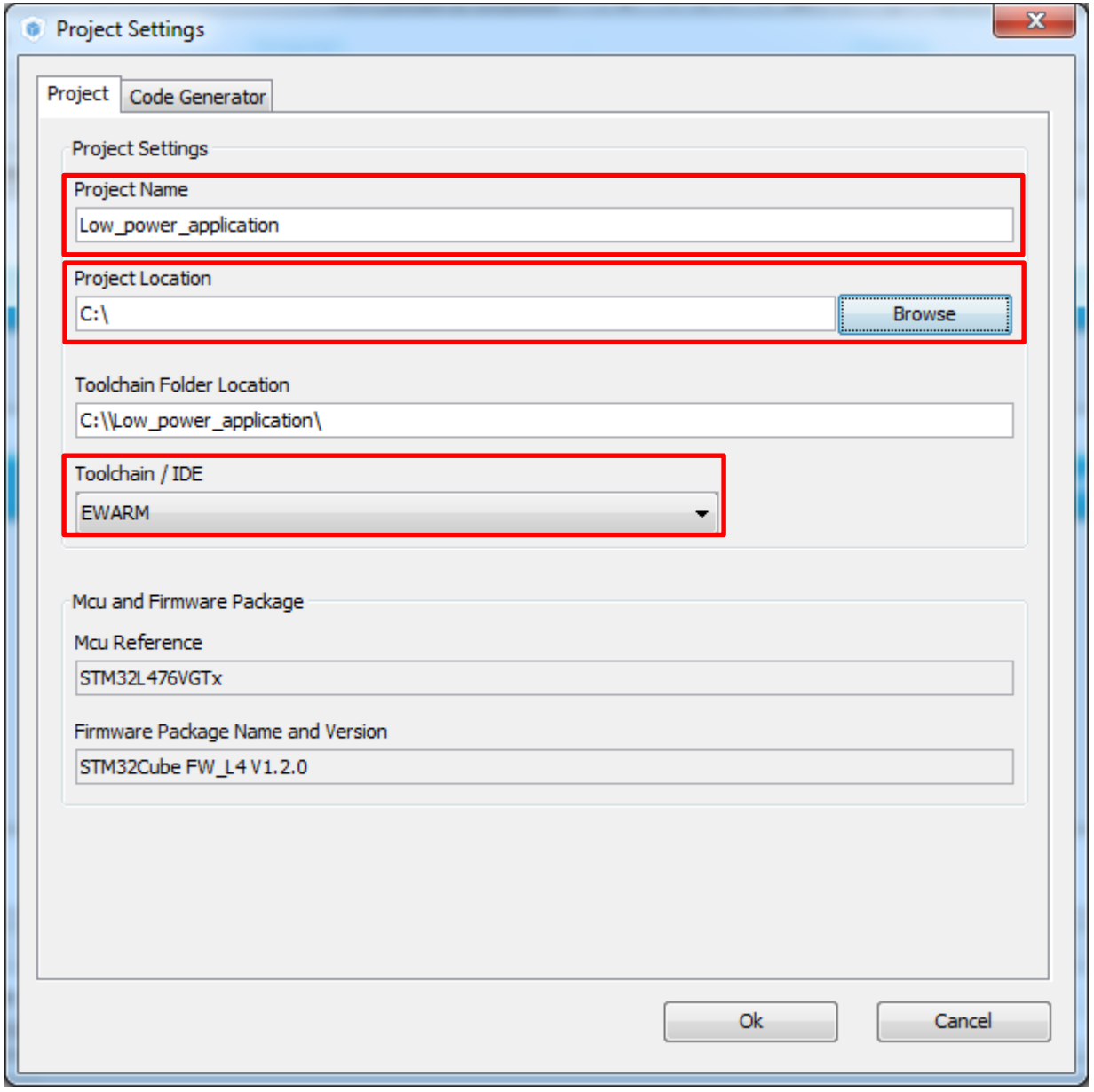


life.augmented

Low power application

- Source code generation
  - To generate code select from Menu:

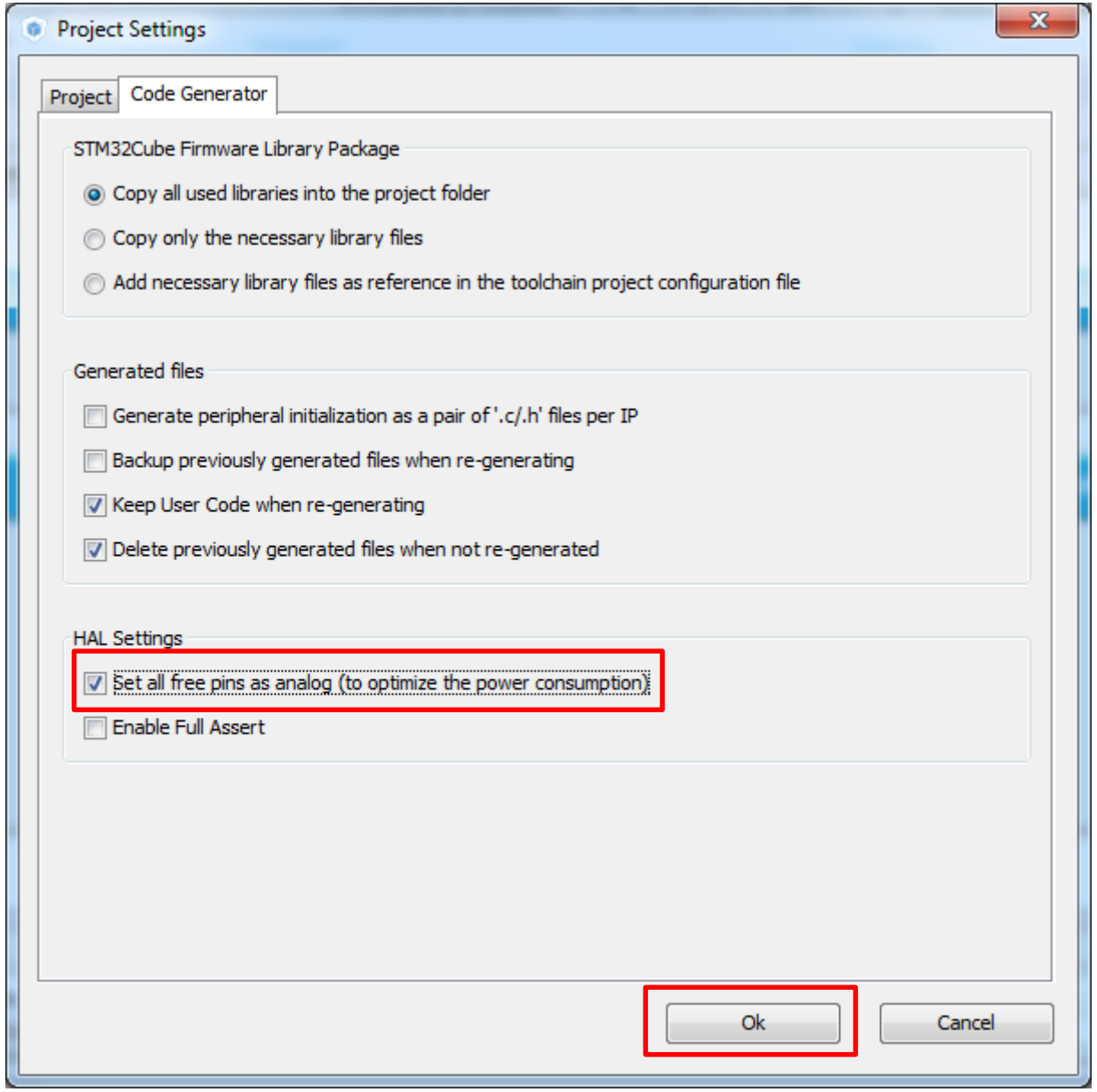
    Project > Generate Code
  - Fill in the project name
  - Fill in the project location
  - Select type of toolchain



Low power application

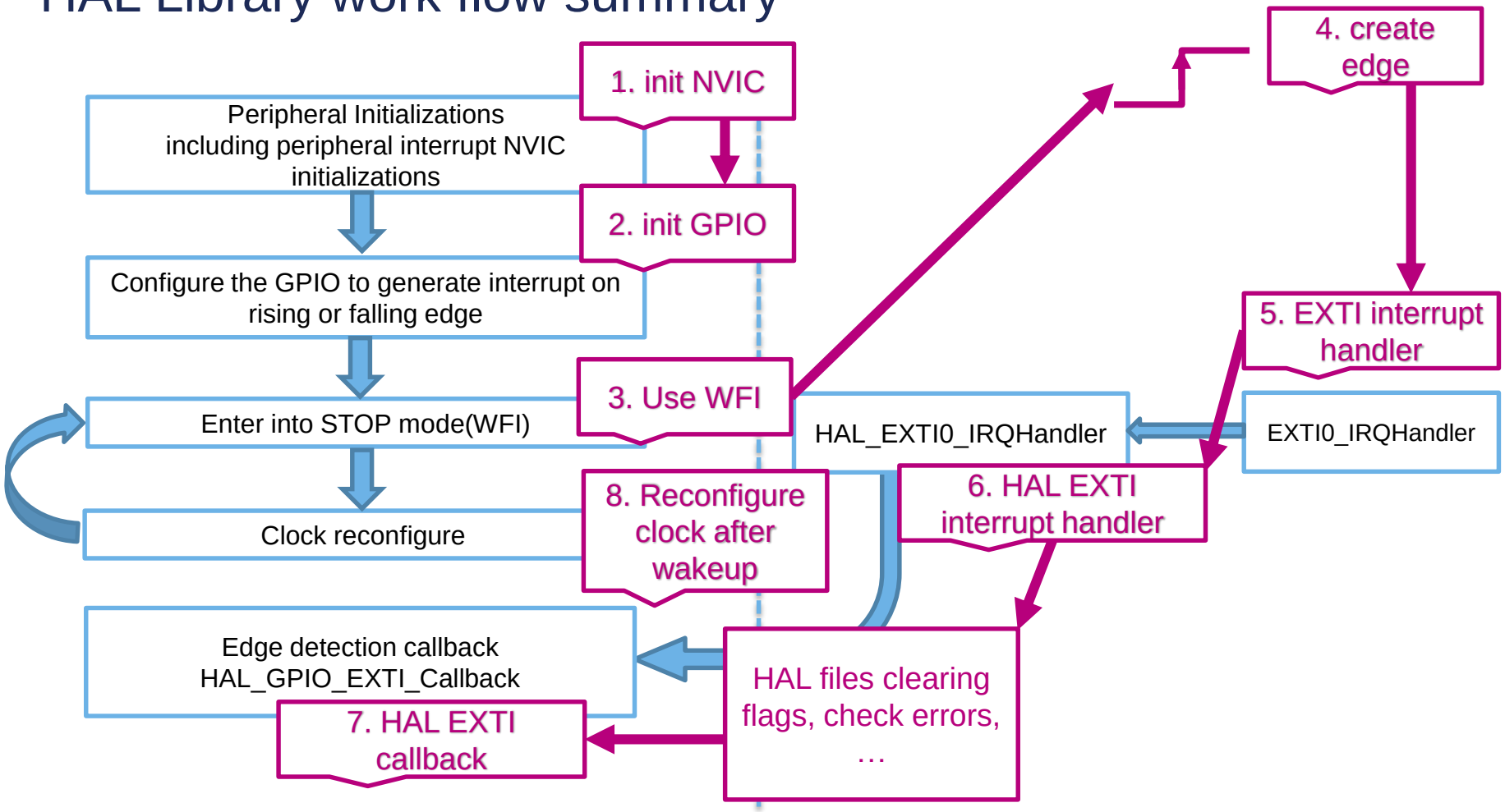
- Source code generation
  - To decrease current consumption select

    Set all free pins as analog
  - Click **OK** to accept



Low power application

HAL Library work flow summary

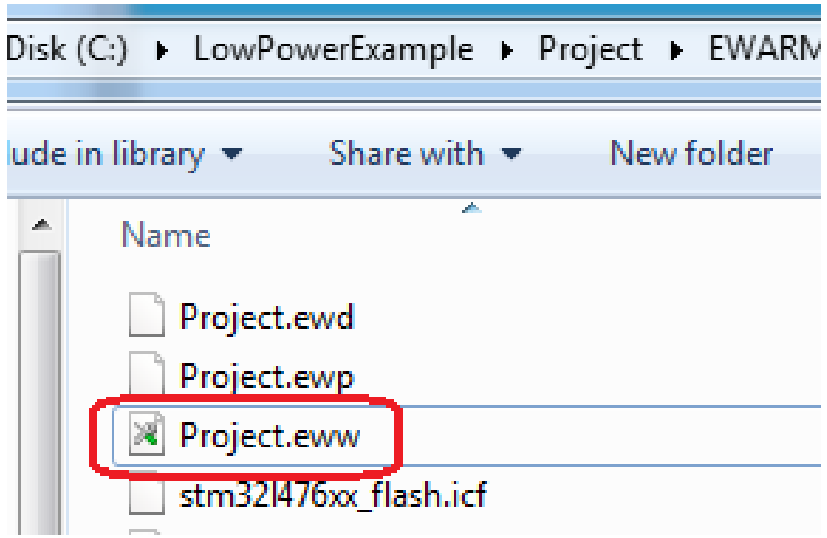


life.augmented

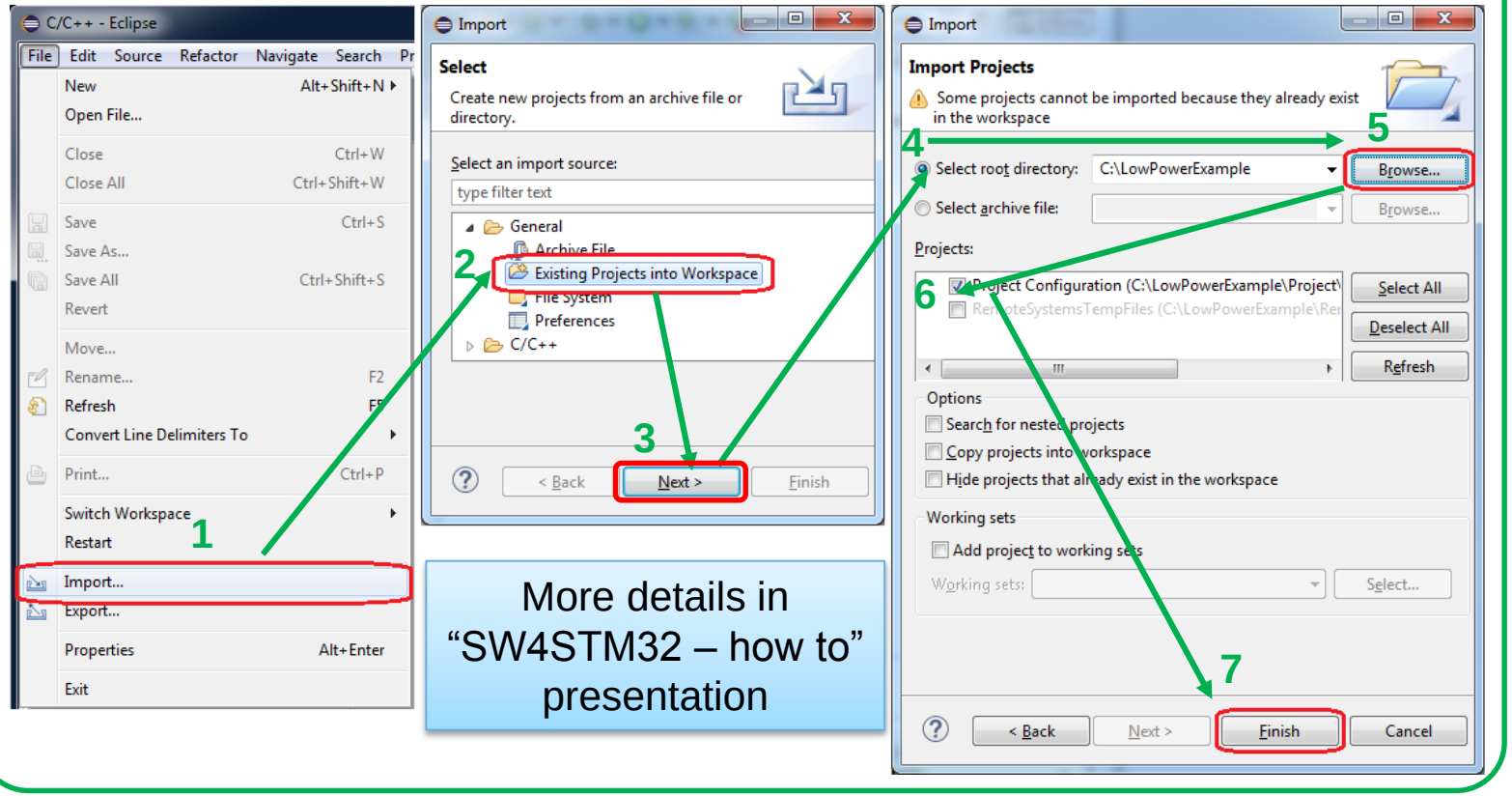
Low power application

- Opening of newly generated project code with IAR is straightforward – it is enough to click on project file and rest will be done by GUI itself (left slide below)
- It is “a bit” more complex with SW4STM32 however – please follow the right slide below for guidance…

EWARM



SW4STM32



life.augmented

Low power application

- Open project in IDE and put source code between */* USER CODE BEGIN … */* and */* USER CODE END … */* tags in ***main.c*** file

1) Switch off unused peripheral clocks in main() function
   - Unused GPIOs: GPIOB, GPIOC, GPIOD, GPIOH

2) Implement STOP2 mode in while(1) infinite loop
   - Suspend Systick to prevent it from waking up MCU
   - Enter STOP2 mode
   - Resume Systick after MCU leaves STOP2 mode

```
/* USER CODE BEGIN 2 */

  __HAL_RCC_GPIOB_CLK_DISABLE();
  __HAL_RCC_GPIOC_CLK_DISABLE();
  __HAL_RCC_GPIOD_CLK_DISABLE();
  __HAL_RCC_GPIOH_CLK_DISABLE();

/* USER CODE END 2 */
```

```
/* USER CODE BEGIN WHILE */
while (1)
{
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
HAL_SuspendTick();
HAL_PWREx_EnterSTOP2Mode(PWR_STOPENTRY_WFI);
HAL_ResumeTick();
}
/* USER CODE END 3 */
```

Low power application

- Put source code between */* USER CODE BEGIN … */* and */* USER CODE END … */* tags in ***stm32l4xx_it.c*** file

1) Change the LED state
   - Toggle PE8 pin

```
void EXTI0_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI0_IRQn 0 */

  /* USER CODE END EXTI0_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
  /* USER CODE BEGIN EXTI0_IRQn 1 */
  HAL_GPIO_TogglePin (GPIOE, GPIO_PIN_8);
  /* USER CODE END EXTI0_IRQn 1 */
}
```

Low power application

- Verify application behaviour
  - LED toggle after each button press
- Verify current consumption
  - Connect ammeter to JP5



The End



www.st.com/stm32l4

life.augmented